П.В. НОВИКОВ

ЛОГИЧЕСКОЕ ПРОГРАММИРОВАНИЕ
на Visual Prolog 5.2

Предисловие

Настоящее пособие содержит описание цикла лабораторных работ, предназначенных для изучения логического программирования на языке ПРОЛОГ в версии Visual Prolog 5.2. Материал, представленный в пособии, базируется на сведениях, изложенных в ранее изданных печатном [1] и электронном [2] пособиях автора. При этом максимально широкие сведения по программной оболочке Visual Prolog 5.2 имеются в [3]. В языке ПРОЛОГ реализована логика исчисления предикатов 1-го порядка. Любая версия транслятора языка ПРОЛОГ, в том числе Visual Prolog 5.2, содержит механизм автоматического поиска решений, представляющий собой встроенный алгоритм резолюции вместе с алгоритмом унификации термов. Логические программы имеют особый, декларативный вид, который соответствует логической парадигме. Этой парадигме соответствуют такие синтаксические конструкции, как клаузы, факты, правила, термы, предикаты динамических баз данных, альтернативные доменные структуры, левые и правые рекурсии, красные и зелёные отсечения, и т.д. Для освоения синтаксиса языка Visual Prolog 5.2 в предлагаемом цикле работ все основные программные конструкции последовательно изучаются. Это предоставляет обучающемуся возможность освоить приёмы логического программирования и приобрести необходимые знания, умения и навыки в этой парадигме программирования. Логическое программирование располагает весьма внушительными возможностями по решению задач символьной обработки информации, в том числе задач так называемого искусственного интеллекта, которым посвящены отдельные лабораторные работы.

Р а б о т а № 1. **Основные синтаксические конструкции языка Visual Prolog 5.2**

Цель работы - знакомство с фактами и правилами, основными синтаксическими конструкциями логических программ. Изучение главных компонент логических программ на языке Visual Prolog 5.2.

Общие сведения

Все логические программы состоят из так называемых ***клауз***, которые, фактически, являются логическими утверждениями [4]. Клаузы (или, говоря по-другому, клозы [5]) могут состоять из посылки и заключения, связанных импликацией (то есть посылка влечёт заключение). Пример:

«*Если геометрическая фигура – ромб, то она – параллелограмм*».

или

«*Каждый умеет плавать, если служит матросом*»

В логическом программировании принято посылку называть **телом** клаузы, а заключение утверждения называть **головой** клаузы.

Посылка утверждения в логическом программировании может быть элементарной или сложной. Сложная посылка - конъюнкция элементарных посылок. Всякая элементарная посылка выражается в логической программе предикатом, являясь с позиции формальной логики атомарным (неделимым) и элементарным (простейшим) суждением.

Теоретически клауза есть конструкция [6]:

$$A_1, A_2, \ldots, A_n \leftarrow B_1, B_2, \ldots, B_m.$$

Здесь $A_1, A_2, \ldots, A_n$ – дизъюнкция заключений, а $B_1, B_2, \ldots, B_m$ – конъюнкция посылок, где A_i и B_j – предикаты.

Если m = 0, то такая клауза называется **факт**:

$$A_1, A_2, \ldots, A_n \leftarrow.$$

Если m> 0 и n> 0, то такая клауза называется **правило**.

Если m> 0 и n = 0, то такая клауза является **вопросом** (*запросом*) к логической программе и записывается отдельно от других клауз в разделе **goal**.

Если m = 0 и n = 0, то такая клауза называется **пустая клауза**:

$$\leftarrow.$$

Успешный поиск решения задачи алгоритмом резолюции всегда завершается пустой клаузой. Если поиск решения задачи алгоритмом резолюции не завершается пустой клаузой, то транслятор Visual Prolog 5.2 завершает работу сообщением “No Solution!”.

В логических программах всегда n = 1, то есть в логической программе правила имеют вид:

$$A \leftarrow B_1, B_2, \ldots, B_m.$$

Факты в логической программе имеют вид:

$$A \leftarrow .$$

Это клаузы Хорна (хорновские дизъюнкты) [4, 6].

При этом более общая клауза

$$A_1, A_2, \ldots, A_n \leftarrow B_1, B_2, \ldots, B_m.$$

может быть заменена на n хорновских [6]:

$$A_1 \leftarrow B_1, B_2, \ldots, B_m.$$
$$A_2 \leftarrow B_1, B_2, \ldots, B_m.$$
$$\ldots$$
$$A_n \leftarrow B_1, B_2, \ldots, B_m.$$

Предикаты A_i и B_j это логические функции, принимающая значения «истина» (true) или «ложь» (false) в зависимости от их аргументов, имеющих произвольный тип (кроме логического) и любое количество.

Логическая программа является конъюнктивной нормальной формой [5, 7-8], где элементарные дизъюнкции представляют собой хорновские дизъюнкты, то есть клаузы [6].

В логической программе на языке Visual Prolog 5.2 все предикаты, из которых состоят факты и правила, описываются в разделе predicates (где перечисляются имена предикатов с типами всех аргументов, от которых предикаты зависят), а сами факты и правила располагают в разделе clauses, следующем после раздела predicates.

Утверждение: «число - круглое, если нацело делится на пять» в виде клаузы выглядит так: $A \leftarrow B.$

Здесь голова клаузы A - «число - круглое», выражено предикатом *круглое* (X), а тело клаузы B - «число нацело делится на пять» выраженно предикатом *делится_на_пять* (X), то есть:

$$круглое\ (X) \leftarrow делится_на_пять\ (X).$$

При переводе на английский это правило имеет вид, принятый в логических программах:

$$round(X)\ \text{if}\ divide_by_five(X).$$

В то же время утверждение

«Пифагор - математик»

является фактом вида:

математик («Пифагор») ←

и в логической программе становится фактом:

mathematic («Пифагор»).

Аналогично, предложение

«точка пересечения прямых x и y есть Z»

представляется клаузой

точка_пересечения_прямых (x, y, 'Z') ←

и является фактом:

point_of_lines_intersection (x, y, 'Z').

Логическую функцию-связку *конъюнкция* в теле правила записывают как and или заменяют запятой «,». В программах допускается как запятая, так и общеизвестное and.

Логическая функция *импликация* в теории задана символом ←, а в логической программе парой символов :- или словом if. Предложение «*Фигура - квадрат, если она - прямоугольник и ромб*» выглядит так:

квадрат(Y) ← ромб(Y), прямоугольник(Y).

В программе ему соответствует правило:

square(Y) if rhombus(Y) and rectangle(Y).

или

square(X) :- rhombus(X), rectangle(X).

Предложение может имеет сложное заключение, например [2]:

«*Если DU есть производная функции U по переменной X,*
и DV есть производная функции V по переменной X, то
либо (DU+DV) есть производная функции (U+V) по переменной X,
либо (DU-DV) есть производная функции (U-V) по переменной X,
*либо (DU*V+U*DV) есть производная функции (U*V) по переменной X,*
*либо (DU*V-U*DV)/(V*V) есть производная функции (U/V) по переменной X*».

В клаузальной форме представляется в виде:

производная_функции (U+V)_по_переменной_X_есть_(DU+DV),
производная_функции (U-V)_по_переменной_X_есть_(DU-DV),
производная_функции(U*V)_по_переменной_X_есть_(DU*V+U*DV),
производная_функции (U/V)_по_переменной_X_есть_(DU*V-U*DV)/ (V*V))
← производная_функции_U_по_переменной_X_есть_DU,
производная_функции_V_по_переменной_X_есть_DV.

А в логической программе будет следующим набором из четырёх клауз:

```
diff (plus (U, V), X, plus (DU, DV)) :- diff(U, X, DU), diff(V, X, DV).
diff (minus (U, V), X, minus (DU, DV)) :- diff(U, X, DU), diff(V, X, DV).
diff (mult (U, V), X, plus (mult (DU, V), mult (U, DV))) :- diff(U, X, DU),
                                                    diff (V, X, DV).
diff (div (U, V), X, div (minus (mult (DU, V), mult (U, DV)), mult (V, V))) :-
                                                    diff (U, X, DU), diff (V, X, DV).
```

Здесь plus (,), minus (,), mult (,) и div (,) есть функторы, означающие действия сложение, вычитание, умножение и деление аргументов (см. работу №2).

Если посылки связаны дизъюнкцией, то предложение представляется несколькими клаузами с одной и той же головой, но разными телами. Предложение «Ребёнком у человека является его сын или его дочь» выглядит как

ребёнок (X, Y) ← сын (X, Y).
ребёнок (X, Y) ← дочь (X, Y).

На Visual Prolog 5.2 можно дизъюнктивное соединение посылок в теле правила соединить с помощью логической функции-связки or (или символа точки с запятой «;»):

```
child (X, Y) if daughter (X, Y) or son (X, Y).
```

или

```
child (X, Y) :- daughter(X, Y); son (X, Y).
```

В разделе **predicates** в программе на Visual Prolog 5.2 перечисляются предикаты с указанием типов и количества их аргументов. Имена предикатов состоят из латинских букв, цифр и символа подчёркивания и начинаются с латинской буквы в любом регистре.

Переменные в логической программе, в отличие от других языков программирования, не хранят значения, но существует для связи информации в клаузе. По этой причине в каждой клаузе одна и та же переменная должна встречаться не менее двух раз. Имена переменных в Visual Prolog 5.2 всегда начинаются с заглавных латинских букв и содержать латинские буквы, цифры и символ подчёркивания: X1, Y2, Ball, P11, B_2.

На каждую переменную в клаузе по умолчанию действует квантор общности (всеобщности) ∀, означающий «для всех» (или «для каждого»).

В логических программах особый смысл имеет анонимная переменная, обозначаемая символом «подчёркивание» ‘_’. Она предназначена для замены однократно встречающихся в клаузе переменных. Квантор существования ∃ в логических программах отсутствует. Если квантор существования ∃ действует на переменную в теле клаузы, то эта переменная заменяется анонимной переменной.

Например,

«Мужчиной является тот, кто является кому-либо отцом».

Переформулированная в логике исчисления предикатов эта фраза звучит:

«Для каждого А существует В такой, что если А - отец В, то А - мужчина».

В классическом исчислении предикатов 1-го порядка выглядит так:

$$(\forall x)\,(\exists y)\ \text{отец}\,(x, y) \rightarrow (\forall x)\ \text{мужчина}\,(x).$$

В клаузальной форме логики эта фраза выглядит так:

$$\text{мужчина}\,(A) \leftarrow \text{отец}\,(A, B).$$

В логической программе соответствующая клауза имеет вид:

man(A) if father (A, _).

Если анонимные переменные расположены в голове клаузы, то их появление эквивалентно действию квантора всеобщности. Пример: предложения

«Каждый ест»

«У всякого есть обувь»

или

«Каждый ошибается»,

представляются в классическом исчислении предикатов 1-го порядка как

$$(\forall u)\ \text{ест}(u),$$

$$(\forall v)\ \text{имеет}\,(v, \text{обувь}),$$

$$(\forall w)\ \text{ошибается}(w)$$

фактами в клаузальной форме:

$$\text{ест}(u) \leftarrow$$

$$\text{имеет}\,(v, \text{обувь}) \leftarrow$$

$$\text{ошибается}(w) \leftarrow$$

и фактами в логической программе:

eats (_).

owns (_, shoes).

faulter (_).

Расположение клауз в программе на Visual Prolog 5.2 может быть произвольным, но клаузы с одной и той же головой должны находиться вместе, не перемешиваясь с другими клаузами. Это ускоряет выполнение программ на языке Visual Prolog 5.2.

В итоге логическая программа может быть рассмотрена как некая база знаний. При этом факты составляют базу данных по решаемой задаче, а правила составляют систему запросов, наилучшую по результативности из всех теоретически возможных. Запросы (вопросы) к логической программе формируются в разделе **goal**, который должен быть перед разделом clauses.

Каждый запрос представляет собой конъюнкцию целей и по форме является телом клаузы без головы. Так, вопрос

«Кто является внуком Петра III?»

задаётся клаузой

← внук (Vnuk, «Пётр_III»).

или запросом в разделе **goal.**

goal: grandson (Vnuk, «Пётр_III»).

Основные встроенные типы данных языка Visual Prolog 5.2 [3]:

char - произвольный символ: ’S’, ’Ф’, ’%’, ’?’, ’\13’, ’*’

string - произвольная строка символов, заключённых в кавычки: “12345”, ”_перец”, ”F”, ”Hello!”, ”ballon”

(Важно, что ”M” не равно ’M’, так как строка из одной литеры здесь не есть символ)

symbol - начинается со строчной латинской буквы, содержит латинские буквы, цифры и символ подчёркивания ’_’ или произвольный набор символов в кавычках: “apples”, “x”, “b_1”, “var12”, “_Prolog”, “John”.

byte - однобайтовое целое число в диапазоне [0; 255]

short - двухбайтовое целое число в диапазоне [-32768; 32767]

ushort - двухбайтовое целое число в диапазоне [0; 65567]

word - двухбайтовое целое число в диапазоне [0; 65567]

integer - двухбайтовое целое число в диапазоне [-32768; 32767] или четырёхбайтовое целое число в диапазоне $[-2^{31}; 2^{31}-1]$

long - четырёхбайтовое целое число в диапазоне $[-2^{31}; 2^{31}-1]$

ulong - четырёхбайтовое целое число в диапазоне $[0; 2^{32}-1]$

dword - четырёхбайтовое целое число в диапазоне [0; 4294967295]

real - восьмибайтовое вещественное число диапазона +1.7E-307…+1.7E+308 по модулю: 0, -1, 1.5E-3, -2.71E 12.

Приведённый далее пример на Visual Prolog 5.2 иллюстрирует структуру программы и некоторые приёмы создания клауз.

```
                    /* Пример 1. Программа на языке Visual Prolog 5.2 */
predicates /* Раздел, где описываются предикаты */
 nondeterm mother (string, string)
 nondeterm father (string, string)
 nondeterm man(string)
 nondeterm woman(string)
 nondeterm male(string)
 nondeterm female(string)
 nondeterm son (string, string)
 nondeterm grandson (string, string)

goal /* Раздел, где задаётся цель логической программы */
 grandson (Vnuk, «Пётр_III»), write (Vnuk), nl, fail; true.
        % определить, имя внука Петра III, написать его, перейти на следующую
        % строку и продолжить поиск другого внука, пока не будут найдены все

clauses /* Раздел, где описываются клаузы логической программы */
                              /* Далее находятся факты */
mother("Екатерина_II", "Павел_I").    /* Екатерина II - мать Павла I */
mother("Мария_Фёдоровна", "Александр_I").
mother("Мария_Фёдоровна", "Константин").
mother("Мария_Фёдоровна", "Николай_I").
mother("Александра_Фёдоровна ", "Александр_II").
mother("Мария_Александровна ", "Александр_III").
mother("Дагмара_Датская", "Николай_II").
mother("Алиса_Гессенская", "Алексей").
mother("Алиса_Гессенская ", "Анастасия").

father("Пётр_III", "Павел_I").        /* Пётр III - отец Павла I */
father("Павел_I", "Александр_I").
father("Павел_I", "Константин").
father("Павел_I", "Николай_I").
father("Николай_I", "Александр_II").
father("Александр_II", "Александр_III").
father("Александр_III", "Николай_II").
father("Николай_II", "Алексей").
father("Николай_II", "Анастасия").

female("Анастасия"). /* пол Анастасии - женский */
male("Алексей"). /* пол Алексея - мужской */
                              /* Далее находятся правила */
woman(X) if mother (X, _) or female(X).
          % Некто - женщина, если она мать кому-либо или её пол – женский
man(X) if father (X, _) or male(X).
          % Некто - мужчина, если он отец кому-либо или его пол - мужской
son (X, Y) if man(X) and mother (Y, X)
           or man(X) and father (Y, X).
% X - сын Y, если X - мужчина и Y - его мать
          % или если X - мужчина и Y - его отец.
grandson (X, Y) :- man(X), father (Z, X), father(Y, Z);
                   man(X), mother (Z, X), father (Y, Z);
                   man(X), father (Z, X), mother (Y, Z);
                   man(X), mother (Z, X), mother (Y, Z).
```

На языке Visual Prolog 5.2 предикаты, являющиеся головой (или *заголовком*) более одной клаузы в программе, должны быть объявлены как nondeterm.

Запуск логической программы на выполнение на языке Visual Prolog в версии Visual Prolog 5.2 осуществляется кнопкой TestGoal. После этого в новом окне нажать на кнопку Continue Evaluation.

Если в программе есть синтаксические ошибки, то откроется окно Errors (Warnings). Следует определить ошибки, закрыть окно ошибок и исправить ошибки в основной программе.

Если в программе нет ошибок, то после нажатия кнопки TestGoal в окне, где редактируется файл, а затем кнопки Continue Evaluation в следующем окне, нужно нажать кнопку Ok в третьем появившемся окне. Откроется окно с результатами.

При этом раздел goal может быть помещён как перед разделом clauses, так и после раздела clauses.

Задания для лабораторной работы № 1

База знаний «Родственные отношения»

Описав предикаты «мать» и «отец», (mother (…, …), father (..., …)), а также «мужской пол» и «женский пол» (male (…, …), female (..., …)), создать с помощью этих предикатов базу знаний, аналогичную приведённому выше примеру, изобразив её семантическую сеть. К базе знаний составить запросы:

1) ребёнок; 2) родитель; 3) мужчина; 4) женщина; 5) муж; 6) жена;
7) мать; 8) отец; 9) сын; 10) дочь; 11) дедушка; 12) бабушка;
13) внук; 14) внучка; 15) брат; 16) сестра; 17) дядя; 18) тётя;
19) племянник; 20) племянница; 21) сводный брат; 22) сводная сестра;
23) ребёнок; 24) родитель; 25) брат по отцу; 26) брат по матери;
27) сестра по отцу; 28) сестра по матери;
29) мужской пол; 30) женский пол;
31) зять (муж дочери); 32) сноха (жена сына);
33) свёкор (отец мужа); 34) свекровь (мать мужа);
35) золовка (сестра мужа); 36) свояченица (сестра жены);
37) деверь (брат мужа); 38) шурин (брат жены);
39) тесть (отец жены); 40) тёща (мать жены); 41) отчим; 42) мачеха;
43) падчерица; 44) пасынок; 45) ятровь (жена брата мужа);
46) кузен (двоюродный брат); 47) кузина (двоюродная сестра);
48) сводный брат; 49) сводная сестра.

Следует составить возможно больше запросов к создаваемой в работе логической программе на основе предложенного выше списка унарных и бинарных предикатов, отражающих родственные отношения. Должно быть минимум семь запросов, не совпадающих с примерами в настоящем учебном пособии.

Рекомендуется использовать либо реальные данные о своих родных, либо о родственных связях известных исторических лиц или литературных персонажей, изобразив генеалогическое дерево родственных отношений.

Факты создаваемой логической программы должны соответствовать изображённому генеалогическому дереву. Если генеалогическое дерево трактовать как граф, то это будет двуцветный орграф, вершины которого есть конкретные лица, а дуги есть отношения «отец» (синие) и «мать» (красные), ведущие от родителя к ребёнку.

Методические указания

При подготовке к выполнению и сдаче этой лабораторной работы рекомендуется дополнительно ознакомиться с теоретическими положениями, изложенными в [5-8]. Редактор языка Visual Prolog 5.2 и встроенные типы данных этого языка программирования можно изучить в [3]. Программа, аналогичная программе «Родственные отношения», рассмотрена в [9].

Содержание отчёта к лабораторной работе № 1

1. Тема работы.
2. Цель работы.
3. Индивидуальное задание на выполнение.
4. Генеалогическое дерево, отображённое в программе (семантическая сеть «Родственные отношения»).
5. Описание предикатов, разработанных программистом.
6. Текст логической программы.
7. Результаты работы программы.
8. Выводы на основе выполненной работы.
9. Список использованных источников (литература).

Р а б о т а № 2. **Структурирование информации на Visual Prolog 5.2**

Цель работы − знакомство с различными способами структурирования информации в логических программах; разработка пользовательских типов данных на языке Visual Prolog 5.2. Разработка альтернативных доменных структур. Изучение встроенного механизма синтеза динамических списковых структур данных. Синтез пользовательских рекурсивных типов данных.

Общие сведения

Структурированное представление информации необходимо для более удобного хранения и обработки этой информации. В логических программах структурированные переменные есть аргументы предикатов. В исчислении предикатов 1-го порядка аргументами предикатов являются термы [1-4, 7]. Понятию **терм** даётся индуктивное определение.

Термом называется символьное выражение:

f (X1, X2, ..., Xn),

где f - имя терма, называемое *функтор* [1-4] или *функциональная буква* [7], а X1, X2, ..., Xn - тоже термы, структурированные или простейшие.

Простейшим термом являются переменная или константное значение. Из определения следует, что структуры могут быть многократно вложены друг в друга. Термы внешне похожи на предикаты, поэтому следует помнить, что предикаты: это логические функции, а термы: это только аргументы логических функций. Функторы есть имена структур. Предикаты не могут быть вложены друг в друга, в отличие от термов. Термы в логическом программировании реализуются с помощью пользовательских типов структурированных данных. Для разработки этих типов в тексте программы на Visual Prolog 5.2 должен быть создан специальный раздел **domains** перед разделом predicates. В разделе domains описываются типы аргументов создаваемых программистом предикатов. Названия этих типов помещаются вместо наименований встроенных типов данных в аргументах предикатов, создаваемых в разделе predicates.

Общий синтаксис создания структурированных типов данных такой:

<имя типа> = <функтор> (<список имён типов аргументов>)

или, в простейшем случае, такой:

<имя типа> = <имя встроенного типа данных>.

В втором варианте это неструктурированный пользовательский тип. Далее, в Примере 2, можно видеть созданные пользователем типы данных. Среди созданных пользовательских типов данных есть простейшие типы: fname, sname, lname, work, day, month и year, и типы-структуры: fio, date, fmember, father, mother и child.

```
/* Пример 2. Структурированные типы данных в Турбо-ПРОЛОГе */
domains % раздел объявления типов данных, созданных пользователем
 fname, sname, lname, work = string
 day, month, year, prices = integer % простые типы
 fio = fio (lname, fname, sname) % структура fio с функтором
 fio date = dat (day, month, year) % структура date с функтором dat
 father = fath (fio, date, work, prices) % тип данных - father
 mother = moth (fio, date, work, prices) % тип данных - mother
 child = chld (fio, date, work, prices) % тип данных - child
predicates
 zapros % предикат для запроса
 family (father, mother, child) % предикат о членах семьи
clauses % клаузы, содержащие информацию о членах трех семей

 family (fath (fio ("Попов", "Иван", "Петрович"),
               dat (10,06,1958), "Инженер", 95000),
         moth (fio ("Сперанская", "Ирина", "Леонидовна"),
               dat (21,09,1961), "Бухгалтер", 9000),
         chld (fio ("Попов", "Сергей", "Иванович"),
               dat (02,03,1979), "Студент", 824)).

 family (fath (fio ("Ковров", "Евгений", "Дмитриевич"),
               dat (29,07,1948), "Экономист", 125000),
         moth (fio ("Коврова", "Ольга", "Геннадьевна"),
               dat (21,09,1954), "Домохозяйка", 0),
         chld (fio ("Ковров", "Олег", "Евгеньевич"),
               dat (7,10,1977), "Грузчик", 70000)).

 family (fath (fio ("Петров", "Денис", "Валентинович"),
               dat (25,11,1952), "Терапевт", 25000),
         moth (fio ("Петрова", "Алла", "Алексеевна"),
               dat (04,02,1952), "Учитель", 15000),
         chld (fio ("Петрова", "Лада", "Денисовна"),
               dat (16,01,1971), "Секретарь", 45320)).

 zapros :- family(_, moth(fio(_,N,_), dat(_,_,Y),_,_),_), Y>1960, write(N), nl, fail;
                                                                    true.
```

Видно, что имя типа может совпадать с функтором (тип fio), но не обязательно. Также видно, что создаваемые структуры могут быть многократно вложены друг в друга. Ведь структура fmember в качестве компонент имеет структурированные типы fio и date, а также простейшие типы work и prices. При этом возможно наличие структурированных типов

данных, имеющих только одну компоненту. Например, типы father, mother и child, фактически лишь добавляют функтор к уже созданному типу fmember. Создание таких типов даёт дополнительные возможности в представлении информации. Между использованием имён типов и функторами в разделе predicates и разделе clauses есть большая разница, так как имена типов используются только в разделе predicates, а функторы - в разделе clauses, при работе со структурированными данными. Наименования типов данных можно делать достаточно длинными, полными и содержательными, так как эти наименования используются в тексте программы небольшое количество раз. Имена же функторов используются многократно. С целью экономии текста программы в разделе clauses рекомендуется имена функторов сокращать. Во вложенных структурах роль главного функтора играет предикат (см. family − структуру в Примере 2).

Важная роль при составлении запросов к структурированной базе данных отведена анонимной переменной («_»). Замена определённых позиций внутри структур анонимными переменными позволяет создать запросы, не выводящие ненужной информации. Одна анонимная переменная (называемая в этом контексте **слотом**) может заменять целую группу объединённых общим функтором переменных. В программе из Примера 2 составлен запрос к структурированной базе данных, выводящий информацию о матерях, рождённых после 1960 года. Наличие слотов позволяет в этом случае игнорировать ненужную информацию как о других членах семей, так и о профессиях и доходах всех мам. Для удобства создан предикат zapros, не имеющий аргументов и фактически являющийся просто логической переменной. Такие нольарные (нольместные, безместные) предикаты упрощают диалог с ПРОЛОГ-машиной, позволяя заранее создать нужные запросы и при необходимости лишь вызывать желаемый запрос без громоздкого набора информации.

Заполнять базу данных можно двумя основными способами:

1) непосредственно с помощью многоаргументных фактов как в приведённом примере программы. Это программирование **с помощью фактов**;

2) посредством бинарных фактов и правил с многоаргументными предикатами. Это программирование **с помощью правил**.

Тогда фраза «Сотрудник Попов Иван Петрович, рождённый 10 июня 1958 г., инженер с окладом 95000 р., с порядковым номером 12», представленная многоместным неструктурированным предикатом, выглядит как:

member (12,"Попов","Иван","Петрович",10,06,1958,"Инженер", 95000).

Девять аргументов этого предиката легко перепутать как при заполнении базы данных информацией, так и при извлечении информации из базы данных

посредством запросов. В структурированном виде эта информация может быть представлена фактом:

```
member (12, fio ("Попов", "Иван", "Петрович"), dat (10, 06, 1958),
                                                work ("Инженер", 95000)).
```

Возможно и иное представление структурированной информации: с помощью нескольких фактов, сделанных с помощью бинарных предикатов:

```
…
name ("Иван", 12).
…
surname ("Попов", 12).
…
fathername ("Иваныч", 12).
…
day (10, 12).
…
month (6, 12).
…
years (1958, 12).
…
work ("Инженер", 12).
…
prices (9500, 12).
…
```

и одного правила для неструктурированного предиката с 10-ю аргументами:

```
member (N, SurNm, Nm, FathNm, D, M, Y, Wrk, Prc, N) :-
                                        name(Nm, N),
                                        surname (SurNm, N),
                                        fathername (FathNm, N),
                                        day (D, N),
                                        month (M, N),
                                        years (Y, N),
                                        work (W, N),
                                        prices (Prc, N).
```

или для одного структурированного предиката с четырьмя аргументами:

```
member (N, fio (SurNm, Nm, FathNm), dat (D, M, Y), work (Wrk, Prc)) :-
                                        name (Nm, N),
                                        surname (SurNm, N),
                                        fathername (FathNm, N),
                                        day (D, N),
                                        month (M, N),
                                        years (Y, N),
                                        work (W, N),
                                        prices (Prc, N).
```

Набор простых фактов с помощью двуместных предикатов с общим для всех фактов аргументом есть набор реляционных таблиц с общим полем (ключевым полем). Такое представление информации более объёмно и требует объявления дополнительных бинарных предикатов в разделе predicates. Однако при этом легко модифицируется, так как заменить один бинарный факт на другой проще, чем многоаргументный факт. Выбор способа структурирования информации выполняется самим программистом. Важно заметить, что система запросов к клаузам, использующим предикаты со структурированными аргументами, не изменяется в зависимости от того, используется база данных со структурированными фактами, или же используется база данных с бинарными фактами и структурированными правилами.

Альтернативные доменные структуры языка Visual Prolog 5.2

Статические структуры не изменяются при работе программы, элементы таких структур имеют фиксированный тип и не имеют возможностей более гибко представлять информацию. В Примере 2 у фактов из базы знаний «семья» существует одна единственная позиция «работа» (work), а также позиция «зарплата» (prices) для любого члена семьи. Это неудобно на практике, так как отразить в рассмотренной базе знаний информацию о неработающих членах семей, об учащихся, о пенсионерах и иждивенцах возможно лишь при заполнении поля work символьными строками. С точки зрения синтаксиса логической программы невозможно обнаружить различий в социальном статусе разных членов семьи. Не всегда удобно строго связывать информацию о положении в семье с социальным положением члена семьи. Отсутствие такой гибкости преодолевает механизм альтернативных доменных структур, имеющийся в языке Visual Prolog 5.2. В Примере 3 дано описание типа данных work, имеющего в одной позиции несколько разных структур с функторами work, study и dependant. Это так называемая *альтернативная доменная структура*, которая формируется следующим образом: вслед за наименованием типа после знака равенства перечисляются через точку с запятой структуры с указанием функторов и типов аргументов, входящих в них:

```
work = work (work_place, post, prices); % структура "работа"
       study (study_place, gruppa, grant); % структура «учёба»
       dependant (status, pension) % структура "иждивенец"
```

Тип данных work описывает позицию в структурированных типах данных

father=fath (fio, date, **work**),

mother=moth (fio, date, **work**),

child=chld (fio, date, **work**),

то есть в одной и той же позиции соответствующей клаузы могут находиться различные структуры данных, например:

fath (fio (), dat (), **work** ("ООП", "Инженер", 19500)),

или

fath (fio (), dat (), **study** ("МАИ", 03419, 1024)),

или

fath (fio (), dat (), **dependant** ("Пенсионер", 2000)).

Так можно формировать гибко структурированную информацию и учитывать различие в типах полей данных на синтаксическом уровне, что позволяет формировать более гибкие запросы вроде

«Как зовут всех неработающих и неучащихся мам?»,

задаваемые в виде:

family (_, moth (Mama, _, dependant (_, _)), _).

Альтернативные доменные структуры, помимо рассмотренных здесь возможностей, позволяют создавать весьма полезные рекурсивные структуры. Это допускает синтаксис языка Visual Prolog 5.2, ведь имя типа поля данных одной из структур в альтернативной записи справа от знака равенства может совпадать с именем типа слева от этого знака равенства. Такой синтаксис позволяет создавать списки и другие динамические структуры данных, как то: бинарные деревья, например, и т. п. Так список данных типа symbol являет собой динамическую бинарную (двухместную) структуру, у которой тип первого элемента symbol, а второй элемент также является списком. Самый простой список - пустой список, где нет совсем элементов (void). Этот список создаётся следующим образом:

```
domains
        list = void;
        l (symbol, list)
```

Здесь в бинарной структуре с функтором l имя самого типа совпадает с именем типа его аргумента. Для работы со списками необходимо их сделать аргументами предназначенных для этого предикатов, например, так:

```
predicates
        p(list)
        q(list)
```

Список [a, b, c, d] может быть представлен фактом

```
p (l(a, l(b, l(c, l(d, void))))).
```

Пустой же список - фактом

```
q(void).
```

Также можно создать «бинарное дерево» и другие рекурсивные типы. При этом часто используемая динамическая структура «список» имеет встроенный механизм создания в языке Visual Prolog 5.2, что значительно облегчает создание списков. Для создания списка, достаточно в разделе **domains** в правой части оператора равенства после описания типа элементов списка поставить знак «*». Тип списков целых чисел с именем number можно создать так:

```
domains
    number = integer*
```

Списки такого типа могут быть аргументами предикатов:

```
predicates
        l(number)
```

Этот механизм предлагает представление списка с помощью квадратных скобок «[» и «]». Элементы списка перечисляются через запятую, а пустой список записывается как «[]». Так, для создания списка целых чисел в разделе clauses может быть задан факт:

```
l ([3, -4, 5, 2, 7, -6, 1, -8, 9, 0]).
```

Другой способ представления таким образом созданных списков есть разделение списка на голову (Head) и хвост (Tail), разделяемых вертикальной чертой «|»:

```
l ([Head | Tail]).
```

Это удобно при синтезе правил обработки списков, которые подробно представлены в следующей главе.

В Примере 3 список состоит из структурированных элементов:

```
domains
    child= chld (fio, date, work)
    children=child*
```

```
    /* Пример 3. Альтернативные доменные структуры и списковый тип */
domains
 day, month, year, price, grant, pension, gruppa=integer
 fname, sname, lname, work_place, study_place, post, status=string
 fio=fio (lname, fname, sname)
 date=dat (day, month, year)
 work=work (work_place, post, price);        % структура "работа"
          study (study_place, gruppa, grant); % структура "учёба"
          dependant (status, pension)      % структура "иждивенец"
 father=fath (fio, date, work)
 mother=moth (fio, date, work)
 child=chld (fio, date, work)      % элементы списка имеют этот тип
 children=child* % списковый тип children («дети»)
predicates
 zapros1
 zapros2
 family (father, mother, children)
clauses
family (fath (fio ("Васильев", "Иван", "Никифорович"), dat (10,06,1958),
              work ("ООП", "Инженер", 145000)),
        moth (fio ("Васильева", "Ирина", "Леонидовна"), dat (21,09,1961),
              work ("ПБОЮЛ Васильева", "Продавец", 12000)),
% далее следует список из двух структур с информацией о двух детях
       [chld (fio ("Васильев", "Сергей", "Иванович"), dat (02, 03, 1979),
              study ("МАИ", 13, 324)),
        chld (fio ("Васильев", "Алексей", "Иванович"), dat (24, 07, 1983),
              dependant ("Безработный", 15000))]).
family (fath (fio ("Козлов", "Евгений", "Дмитриевич"), dat (29, 07, 1975),
              work ("ТОО Дружба", "Экономист", 125000))),
        moth (fio ("Козлова", "Ольга", "Геннадьевна"), dat (21, 09, 1977),
              dependant ("Домохозяйка",0))),
       []). % пустой список
family (fath (fio ("Петров", "Денис", "Валентинович"), dat (25, 11, 1952),
              work ("П-ка №61", "Терапевт", 125000))),
        moth (fio ("Петрова", "Алла", "Алексеевна"), dat (04, 02,1952),
              work ("школа №866", "Учитель", 150000)),
% список из одного структурированного элемента – один ребёнок в семье
       [chld (fio ("Петрова", "Лада", "Денисовна"), dat (16,01,1971),
              work ("СМУ N71 ","Секретарь",4532))]).
zapros1:- % фамилии, имена и отчества всех неработающих мам.
family (_, moth (Mama, _, dependant (_, _)), _), write (Mama), nl, fail;
                                                                  true.
% Вывести информацию обо всех детях.
zapros2:- family(_, _, Baby), write(Baby), nl, fail; true.
```

Задания для лабораторной работы № 2

Часть 1. Создание структурированной базы знаний

Необходимо создать хорошо структурированную базу знаний и систему запросов к ней в соответствии с вариантами задания. При разработке базы знаний для хранения информации можно создать только один предикат.

1. Разработать базу знаний «семья» (family), используя данные о своей семье и предках.
3. Разработать базу знаний «преподаватели» (professor), используя данные о преподавателях.
4. Разработать базу знаний «режим дня» (regime), используя расписание дня типового студента или собственный режим дня.
5. Разработать базу знаний «расписание занятий», используя недельное расписании учёбы или своё расписание.
6. Разработать базу знаний «табель оценок» (tabel) на основе успеваемости группы студентов.
7. Разработать базу знаний «расписание экзаменов» с указанием сведений об экзаменаторе.
8. Разработать базу знаний «сотрудник» (member), содержащую поля «ФИО» (fio), «день рождения» (date), «карьера» (career) и т. п.
9. Разработать базу знаний «анкета» (form), содержащую поля «ФИО» (fio), «адрес» (address), «день рождения» (date), «текущая дата» (currency_date) т. п.
10. Разработать базу знаний «студент» (student), содержащую поля «ФИО» (fio), «место учёбы» (place), «оценки» (marks), «учебные планы» (plans) и т. п.
11. Разработать базу знаний «семья», используя данные о семье учащегося.
12. Разработать базу знаний «расписание занятий», используя собственное недельное расписании учёбы.

Часть 2. Улучшение структурированных баз знаний с помощью альтернативных доменов и списковых структур данных

Необходимо создать базу данных с использованием альтернативных доменных структур и встроенного механизма разработки списков.

1. Разработать базу данных «семья», предусмотрев различные варианты занятости для членов семьи, а также возможность представления семей с различным количеством детей и неполных семей.
2. Разработать базу данных «анкета», содержащую альтернативные структуры данных и списки «семейное положение» и «социальный статус».
3. Разработать базу данных «сотрудник», содержащую альтернативные структуры «образование» и «квалификация» и списковое поле «карьера».
4. Разработать базу данных «расписание сессии» как список предметов, где зачёты и экзамены с представлены альтернативной структурой.
5. Разработать базу данных «воинские звания и должности», используя альтернативные структуры для должностей или званий в различных родах войск.

Часть 3. Разработка рекурсивных типов данных

1. Разработать рекурсивный тип «многозначное число» - список цифр.
2. Разработать рекурсивный тип «строка» - список символов.
3. Разработать рекурсивный тип «бинарное дерево». Вершина дерева - его элемент и две «ветви» - такие же «бинарные деревья».
4. Разработать рекурсивный тип «текст» - список строк (список списков).
5. Разработать рекурсивный тип «стек» (LIFO).
6. Разработать рекурсивный тип «квадро-дерево». Вершина дерева – элемент, четыре «ветви» - такие же «квадро-деревья».
7. Разработать рекурсивный тип «цепная дробь» - список коэффициентов.
8. Разработать рекурсивный тип «полином» - список коэффициентов.
9. Разработать рекурсивный тип «очередь» (FIFO).
10. Разработать рекурсивный тип «префиксное алгебраическое выражение».

Методические указания

При подготовке к выполнению задания рекомендуется ознакомиться с теоретическими понятиями *функтор* (функциональная буква) [7] и *терм*, изложенными также в [1-3], а также освоить альтернативные структуры данных по [1-3]. Списковые типы так же, как и альтернативные домены, описаны в [1-3, 5, 9-11]. Встроенные предикаты обработки списков также рассмотрены в [1-3, 5, 9-11]. Различные динамические структуры данных и способы их организации так же, как примеры использования структур, отличных от простейших баз данных, можно изучить по [9]. Заполнение базы данных с использованием бинарных предикатов рассмотрено в [10]. Структурированные типы данных у языка Visual Prolog 5.2 описаны в [3].

Содержание отчёта по 2-й работе:

1) заголовок лабораторной работы;
2) цель лабораторной работы;
3) индивидуальное задание;
4) исходные данные для структурирования;
5) описание предикатов, созданных программистом;
6) текст логической программы;
7) результаты работы программы (один запрос должен выводить все данные);
8) выводы;
9) список использованных источников (литература).

Р а б о т а № 3. **Различные виды рекурсивных логических программ**

Цель работы - изучение возможностей создания рекурсивных логических программ для осуществления арифметических и алгебраических вычислений на Visual Prolog 5.2, а также для осуществления символьных вычислений на примере списков на Visual Prolog 5.2. Освоение принципов разработки рекурсивных программ обработки списков.

Общие сведения

Рекурсивные функции являются основным средством организации повторяющихся вычислений в логическом программировании. Все версии языка ПРОЛОГ, в том числе Visual Prolog 5.2, допускают возможность создания рекурсивных правил, имеющих следующий общий вид:

<имя правила рекурсии> :- *<список предикатов>*,
<предикат условия выхода из рекурсии>,
<список предикатов>,
<имя правила рекурсии>,
<список предикатов>.

Из представленного описания видно, что отличием рекурсивного правила от нерекурсивного правила является использование одного и того же предиката как в голове, так и в теле правила. Такой синтаксис рекурсивного правила приводит к повторным обращениям к этому правилу при вызове. Это позволяет организовывать повторяющиеся вычисления. Чтобы рекурсия не была бесконечной, обычно рекурсивное правило содержит *предикат условия выхода*, останавливающий повторные обращения к рекурсии. Каждый входящий в рекурсивное правило *<списков предикатов>*, может быть в зависимости от задачи произвольным, в том числе и пустым. Рекурсивное правило обязательно используется в тексте логической программы с фактом, имеющем туже голову, что у рекурсивного правила. Этот факт определяет точку останова рекурсии. К этой точке направляются рекурсивные вычисления. С этой точкой напрямую связано условие выхода из рекурсии. Для составления рекурсивной программы следует представить желаемую функцию в *рекуррентном* виде. Тогда текст рекурсивного правила будет очевиден. Например, функция «факториал» может выглядеть так:

$$1! = 1,$$

$$n! = (n-1)! * n.$$

Клаузы логической программы fact «факториал» будут иметь вид:

```
fact (1,1). % точка останова рекурсии
fact (N, F) :- N>1, % условие выхода из рекурсии
        N1=N-1, % вычисление предыдущего шага рекурсии
        fact (N1, F1), % рекурсивный вызов в теле правила
        F=F1*N. % вычисление искомого значения
```

Направление рекурсии в сторону, противоположную точке останова, является типичной ошибкой при создании рекурсивного правила. Это приводит к бесконечной рекурсии. В случае вычисления факториала эта ошибка случится, если переход к следующему шагу рекурсии произойдёт не с убыванием номера (N1=N-1), а с возрастанием номера (N1=N+1). Дабы рекурсия не была бесконечной, её всегда направляют в точку останова, то есть в сторону уменьшения рассогласования между текущими значениями аргументов и значениями аргументов в точке останова.

В теле рекурсивного правила имя правила порой встречается более одного раза. Такая рекурсия называется ***многократной*** рекурсией. Например, при вычислении чисел Фибоначчи рекурсивное правило будет дважды рекурсивным, то есть это ***двукратная*** рекурсия:

```
fib (1,1).
fib (2,1).
fib (N, F) :- N>2, N1=N-1, fib (N1, F1), % первый рекурсивный вызов
        N2=N-2, fib (N2, F2), % второй рекурсивный вызов
        F=F1+F2.
```

Также ещё бывает, что одно рекурсивное правило вызывает в своём теле другое рекурсивное правило. Это ***вложенная*** рекурсия. В Примере 4 показаны программы вычисления «фигурных» чисел. Это треугольные и пирамидальные числа. Рекурсивная клауза для вычисления пирамидальных чисел содержит вызов предиката треугольных чисел, то есть осуществляется вложенное рекурсивное вычисление. В примерах рекурсий использованы не рассматриваемые ранее синтаксические конструкции, хорошо известные в других языках программирования, но необычные для ПРОЛОГа: предикаты сравнения из Таблицы 1 и арифметические операции из Таблицы 2.

Таблица 1. Бинарные предикаты сравнения языка Visual Prolog 5.2

=	«равно»
>	«больше»
<	«меньше»
>=	«больше или равно»
<=	«меньше или равно»
<>	«не равно»

Предикаты сравнения используются в традиционной инфиксной форме записи. Символ, обозначающий имя предиката, стоит между аргументами, а не в префиксной форме, когда аргументы располагаются после символа предиката (в логической программе правильней была бы запись <(5,7), означающая 5<7).

Таблица 2. Арифметические операции в языке Visual Prolog 5.2

а) бинарные

+ «сложение»
- «вычитание»
* «умножение»
/ «деление»
div «целая часть от деления»
mod «целый остаток от деления»

б) унарные

+ «унарный плюс»
- «унарный минус»

Операции *присвоение* в логическом программировании нет. Как и где можно сохранить накопленные при работе логической программы данные, обсуждается далее в других лабораторных работах. Например, нужное значение для N1 подставляется в логическое выражение N1=N-1, после чего инфиксный предикат = возвращает значение true. По этой причине изолированное использование арифметических операций в логических программах невозможно. Лишь в логических вычислениях. Также точно можно использовать алгебраические функции из Таблицы 3. Алгебраические функции имеют удобный в логических программах префиксный вид.

Таблица 3. Встроенные алгебраические функции языка Visual Prolog 5.2

abs(X)	модуль X ()
sin(X)	синус X
cos(X)	косинус X
tan(X)	тангенс X
arctan(X)	арктангенс X
exp(X)	экспонента X
ln(X)	натуральный логарифм X
log(X)	десятичный логарифм X
sqrt(X)	корень квадратный из X
round(X)	округление X
trunc(X)	усечение (отбрасывание дробной части) X
random(X)	случайное вещественное число X, $0 \le X < 1$
random (X, Y)	случайное вещественное число Y, $0 \le Y < X$

```
% Пример 4. Рекурсивные арифметические программы
predicates
 treug (integer, integer)
 piramid (integer, integer)

goal
  treug (5, T), piramid (5, P).

clauses
treug (1, 1). % стартовое значение "треугольного" числа
treug (X, Y) :- X>1,
                X1=X-1,
                treug (X1, Y1),
                Y=Y1+X.
        % рекурсивное вычисление "треугольного" числа

piramid (1, 1). % стартовое значение "пирамидального" числа
piramid (X, Y) :- X>1,
                  treug (X, Z),
                  X1=X-1,
                  piramid (X1, Y1),
                  Y=Y1+Z.
% рекурсивное вычисление очередного "пирамидального" числа через
% предыдущее с использованием вызова очередного "треугольного" числа
```

Возможны и другие применения рекурсий. В программе из первой лабораторной работы можно поставить задачу синтезировать рекурсивное правило «предок», определив его как

«Предок человека: это его мать или его отец,
либо предок матери или предок отца».

Такая формулировка может быть представлена рекурсивными клаузами [3]:

```
parent (X, Y) :- mother (X,Y);
                 father (X, Y).

parent (X, Y) :- parent (X, Z), mother (Z, Y);
                 parent (X, Z), father (Z, Y).
```

В рекурсивных правилах с головой parent и в им подобных правилах отсутствует в явном виде условие выхода. Поэтому выход из рекурсии определяется правильно заданным направлением рекурсии и точкой останова рекурсии. Видно, что предикат головы правила стоит в теле правила не последним, а левее последнего правила, если тело правила записывать в строку. Такие рекурсивные правила называют ***левыми*** *рекурсиями* (леворекурсивными правилами).

Возможно изменение словесной формулировки «предок»:

«Предок человека: это его мать или его отец,
либо мать предка или отец предка».

В этом случае рекурсивная программа выглядит по-другому [3]:

parent (X, Y) :- mother (X, Y);
father (X, Y).

parent (X, Y) :- mother (X, Z), **parent** (Z, Y);
father (X, Z), **parent** (Z, Y).

Эти две программы **parent** совпадают по результатам, но различны по вычислительным затратам.

Видно, что во второй программе предикат головы правила стоит в теле правила последним, то есть крайним справа, если тело правила записывать в строку. Такие рекурсивные правила называют *правыми* ***рекурсиями*** (праворекурсивными правилами).

Отсутствие условия выхода у программ **parent** приводит к тому, что качество работы этих рекурсивных программ, вступая в противоречие с декларативными принципами логического программирования, становится зависимым от порядка расположения в программе фактов с точками останова и самих рекурсивных правил.

Оба рассматриваемых примера приводят к правильным результатам. Но в случае с леворекурсивными правилами после правильных ответов может появиться сообщение «Stack overflow!», или же иное сообщение о соответствующей переполнению стека ошибке. Таким образом, левая рекурсия в этом случае может привести к бесконечному рекурсивному вычислению из-за того, что в рекурсивном правиле отсутствует предикат условия выхода.

Вариант правой рекурсии, когда факты точек останова стоят после праворекурсивных правил, также приводит к успешным результатам, как и в программе, где факты точек останова стоят перед праворекурсивными правилами. Таким образом, правильные результаты выдаёт следующая программа [3]:

parent (X, Y) :- mother (X, Z), **parent** (Z, Y);
father (X, Z), **parent** (Z, Y).

parent (X, Y) :- mother (X, Y);
father (X, Y).

Однако изменение порядка фактов и правил в леворекурсивном случае не приведёт к успешному выполнению задачи, а в момент запуска сразу выдаст сообщение «Stack overflow!», или же сообщение о соответствующей переполнению стека ошибке без правильных ответов на запрос [3]:

parent (X, Y) :- **parent** (X, Z), mother (Z, Y);
parent (X, Z), father (Z, Y).

parent (X, Y) :- mother (X, Y);
father (X, Y).

Логические программы обработки списков

Рекурсивные программы применяются не только для числовых вычислений, но и для обработки рекурсивных структур данных, и прежде всего, для обработки списков. Нижеследующий материал посвящён программам обработки списков. Динамическая структура «список» весьма важна с практической точки зрения при разработке различного программного обеспечения. Возможности логического программирования в обработке таких типов данных во всех версиях языка ПРОЛОГ на порядок выше возможностей императивных языков программирования и близки к большим возможностям функционального программирования на языке ЛИСП в любой версии.

При изучении приёмов логического программирования динамическая структура «*список*» оказывается очень полезной для демонстрации больших возможностей рекурсивных правил в обработке символьных данных. При этом рекурсивная структура списков удачно сочетается с рекурсивной структурой правил. Встроенный механизм создания списков представлен в работе №2. Теперь для этих списков нужно найти ответы на вопросы, например: о принадлежности элемента списку, о длине списка, о подсписке списка, о соседних элементах списка и т. п. Следует рассмотреть программы добавления элемента в список, удаления элемента из списка, слияния списков, обращения списка, перестановки элементов в списке, сортировку списка, разбиение списка по заданному правилу, проверку упорядоченности списка и т.д., а также решить задачи дублирования элементов списка, выполнения алгебраических операций с элементами списка, и т.п. Список, как говорилось в работе №2, есть бинарная рекурсивная структура, то есть структура, имеющая две составляющие, две компоненты. Первая компонента есть элемент некоторого типа, а вторая компонента есть список того же типа, что и определяемый. Конечная точка такого рекурсивного определения - пустой список []. Первый элемент такой бинарной структуры называется «голова» списка, а второй - «хвост» списка: [Голова| Хвост] или [Head| Tail]. Добавить новый элемент в список следует, сделав его головой нового списка, в то время как список, в который добавляется этот новый элемент, станет хвостом нового списка. Удаление элемента из списка также традиционно осуществляется с помощью головы списка. В итоге создаётся более короткий список, головой которого будет голова хвоста исходного списка.

Разделение на голову и хвост, между которыми стоит вертикальная черта «|», - способ стандартного представления списков на языке Visual Prolog 5.2:

p ([Head| Tail]),

где p(...) - произвольно выбранное имя предиката для хранения списка. Такой синтаксис чаще всего используется в правилах обработки списков, а не в фактах, осуществляющих хранение списков. Для создания спискового типа данных в разделе domains следует в правой части от знака равенства после имени типа элементов списка поставить знак «*»:

domains number = integer*

Создаваемый список должен быть аргументом некоторого предиката: predicates p(number). Механизм создания списков ПРОЛОГа предполагает представление списка данных с помощью стандартной синтаксической конструкции - квадратных скобок «[» и» «]». Элементы перечисляются через запятую, а пустой список записывается как []. Список целых чисел может быть задан факт:

clauses p ([1,2,3,4,5,4,3,2,1,0]).

Возможны списки со структурированными элементами, такие как списки списков дат, каждая из которых является структурой, содержащей данные о дне, месяце и числе:

```
domains
        day, month, year = string
        date = d (day, month, year)
        data = date*
```

При необходимости можно и не пользоваться встроенным механизмом создания списков, а создать такую динамическую структуру самостоятельно, как описано в данной лабораторной работе:

```
domains list=void;
                l (symbol, list)
```

Тогда список [a, b, c, d] может быть представлен фактом

p(l(a, l(b, l(c, l(d, void))))).

Методы обработки списков используют рекурсивные вычисления. Очень важная программа обработки списков − программа append для соединения двух списков в один. Эта рекурсивная программа состоит из факта:

append ([], L, L).

и рекурсивного правила:

append ([Head | As], Bs, [Head | Cs]) :- append(As, Bs, Cs).

Факт означает истинное утверждение: «Соединение первого пустого списка [] со вторым списком L есть второй список L». Правило означает:

«Если результат соединения списка As со списком Bs есть список Cs, то результат соединения списка с головой Head и хвостом As со списком Bs, есть список с головой Head и хвостом Cs».

На своей основе рекурсивной программы append легко и просто создать несколько других нерекурсивных программ обработки списков, называемых программами группы append [1-3, 9].

а) Правило определения элемента списка:

mem_app (El, Lst) :- append(_, [El|_], Lst).

«Переменная El −элемент списка Lst, если список Lst есть соединение произвольного списка со списком, головой которого является элемент El».

б) Правило определения «префикса» заданного списка, то есть списка нескольких первых последовательных элементов заданного списка:

prefix (Ps, Ls) :- append (Ps, _, Ls).

«Список Ps есть префикс списка Ls, если список Ls есть результат соединения списка Ps с произвольным списком».

в) Правило определения «суффикса» списка, то есть списка нескольких последних элементов заданного списка, расположенных друг за другом:

suffix (Ss, Ls) :- append(_, Ss, Ls).

«Список Ss есть суффикс списка Ls, если список Ls есть результат соединения произвольного списка со списком Ss».

г) Правило определения подсписка списка:

sublist (Xs, Ys) :- prefix(Zs, Ys), suffix(Xs, Zs).

«Список Xs есть подсписок списка Ys, если список Xs есть суффикс списка Zs, который является префиксом списка Ys».

д) Правило определения последнего элемента списка:

last (El, Lst) :- append(_, [El], Lst).

«El - последний элемент списка Lst, если Lst есть результат соединения произвольного списка со списком, состоящим из одного элемента El».

е) Правило определения соседних элементов в списке:

adjacent (X, Y, Zs) :- append(_, [X, Y| _], Zs).

«Переменные X и Y соседние в списке Zs, если список Zs есть результат соединения произвольного списка и списка, первые элементы которого есть переменные X и Y».

ж) Леворекурсивное правило обращения списка, вызывающее append:

```
reverse ([], []).
reverse ([Head| Xs], Zs) :- reverse(Xs, Ys), append(Ys, [Head], Zs).
```

«Обращение пустого списка [] есть этот пустой список []».

«Чтобы обратить список с головой Head и хвостом Xs в список Zs, нужно обратить список Xs в список Ys и соединить обращённый список Ys со списком, состоящим из одной переменной Head, в список Zs».

Рекурсивные правила append и reverse показывают эффективность и простоту создания логических программ обработки списков.

Другой пример программы обработки списков: программа print_list:

```
print_list ([]).        % Выход из рекурсии - "пустая" печать пустого списка.
print_list ([Head| Tail]) :-        % Для печати элементов списка в столбик:
                    write (Head),           % напечатать "голову" списка,
                    nl,                     % перейти на другую строку и
                    print_list (Tail).      % напечатать "хвост" списка.
```

Рекурсивная программа для определения принадлежности элемента списку:

```
member (El, [El| _]).           % "Голова" списка принадлежит своему списку
member (El, [_| Tail]):- member(El, Tail).
```

«Элемент принадлежит хвосту списка, значит принадлежит всему списку.»

Рекурсивная программа удаления первого вхождения элемента в список:

```
select (Elem, [Elem| Tail], Tail).          % Удаление элемента из головы списка.
select (Elem, [Head| Xs], [Head| Ys]) :- select(Elem, Xs, Ys).
```

«Если список Ys - результат удаления Elem из списка Xs,
то [Head| Ys] есть результат удаления Elem из списка [Head| Xs].»

В Примере 5 показаны логические программы для операций со списками, созданные с помощью встроенного механизма создания списков.

В Примере 6 показано, что логические программы могут быть успешно применены и для списковых структур данных, созданных пользователем. Дополнительно в этом примере рассмотрен механизм использования встроенного предиката findall. Это предикат позволяет формировать список во время выполнения программы из набора одноимённых фактов (из реляционной таблицы).

Предикат **findall** имеет три аргумента. Первый аргумент − переменная, из значений которой будет состоять формируемый список. Второй аргумент - предикат, имя которого определяет реляционную таблицу фактов. В предикате должна быть указана позиция переменной, имя которой есть первый аргумент. Третий аргумент предиката findall - выходной список, тип элементов которого совпадает с типом первого аргумента. Предикат findall является предикатом второго порядка, так как его аргумент − тоже предикат.

```
        /* Пример 5. Программы обработки списков на Турбо-ПРОЛОГе */
domains
  element=integer % Элементы списка - целочисленные.
  list=element*      % Тип list - список заданных элементов.

predicates
 q (integer, element)          % Предикат для хранения элемента списка
 p(list)                  % Предикат для хранения заданного списка
 add (element, list, list)      % Предикат добавления элемента в список
 select (element, list, list)   % Предикат удаления одного вхождения элемента
 append (list, list, list)      % Предикат слияния двух списков в один.
 print_list(list)               % Предикат для печати списка.
 member (element, list)         % Предикат принадлежности элемента списку.
 reverse (list, list)           % Предикат перестановки элементов в списке.
 rul(list)                      % Предикат для демонстрации работы findall.

clauses
 p ([1,0,2,9,3,8,4,7,5,6]).  % Аргумент предиката - список целых чисел.

 add (Head, Tail, [Head| Tail]).    % Добавление элемента в голову списка.

 select (Elem, [Elem| Tail], Tail).
 select (Elem, [Head| Xs], [Head| Ys]) :- select(Elem, Xs, Ys).

 append ([], List, List).
 append ([Head| Xs], Ys, [Head| Zs]) :- append (Xs, Ys, Zs).

 print_list ([]).
 print_list ([Head| Tail]) :- write (Head),
                              nl,
                              print_list (Tail).
member (El, [El| _]). %
member (El, [_| Tail]):- member(El, Tail).


reverse ([], []).
reverse ([Head| Xs], Zs) :-       % Для обращения списка нужно:
        reverse (Xs, Ys),         % обратить его "хвост" и
        append (Ys, [Head], Zs). % соединить обращённый список с "головой".

 /* Демонстрация работы встроенного предиката findall. */

q(1,12).
q(2,11).
q(3,10).       % Факты, хранящие однотипную информацию.

rul (List_1_2) :-          % Правило для демонстрации работы предиката findall,
        findall (Elem1, q (Elem1, _), List_Elem1),    % список первых аргументов
        findall (Elem2, q (_, Elem2), List_Elem2),    % список вторых аргументов
        append (List_Elem1, List_Elem2, List_1_2), % соединение обоих списков
        print_list (List_1_2). % печать итогового списка.
```

%Пример 6. Рекурсивная структура *список*, программы обработки списков

domains

```
element=integer % элемент списка задан как целочисленный
list= void;          % список - либо пустой (void), либо
      l (element, list) % бинарная рекурсивная структура, где первый
                        % элемент заданного типа, а второй – список
```

predicates

```
add (element, list, list) % добавление элемента в "голову" списка
print_list(list) % поэлементная печать списка
append (list, list, list) % соединение двух списков в один
select (element, list, list) % удаление одного вхождения элемента из списка
member (element, list) % проверка принадлежности элемента к списку
reverse (list, list) % обращение списка в обратном порядке
p(list) % предикаты для задания списков
q(list)
query1 % предикаты для проверочных запросов
query2
query3
query4
query5
```

clauses

```
p (l(3, l(2, l(1, l(0, void))))).

q (l(5, l(4,void))).

print_list (void) :- nl.
print_list (l(Head, Tail)):- write(Head,' '), print_list(Tail).

append (void, L, L).
append (l(Head, List1), List2, l(Head, List3)) :- append(List1, List2, List3).

add (Head, Tail, l(Head, Tail)).

select (Elem, l(Elem, Tail), Tail).
select (Elem, l(Head, Xs), l(Head, Ys)) :- select(Elem, Xs, Ys).

member (Head, l(Head, _)).
member (Elem, l(_, Tail)) :- member(Elem, Tail).

reverse (void, void).
reverse (l(Head, Xs), Zs):- reverse(Xs, Ys), append(Ys, l(Head, void), Zs).

query1:- p(M), member(3, M).
query2:- p(M), add(4, M, L), print_list(L).
query3:- q(L), p(M), append(L, M, N), print_list(N).
query4:- p(M), reverse(M, L), print_list(L).
query5:- p(M), select(3, M, L), print_list(L).
```

Типы списков, предлагаемые к обработке полученными программами:

1) списки целых чисел типа integer;
2) списки вещественных чисел типа real;
3) списки символов типа char;
4) списки имён типа symbol;
5) списки имён типа string;
6) списки векторов (двумерных);
7) списки векторов (трёхмерных);
8) списки комплексных чисел в алгебраической форме;
9) списки комплексных чисел в тригонометрической форме;
10) списки структур типа data;
11) списки структур типа fio;
12) списки структур типа address.

Задания для лабораторной работы № 3

Часть 1. Рекурсивные арифметические вычисления

1. Вычислить таблицу чисел Фибоначчи рекуррентным способом.

2. Распечатать рекурсивно таблицу факториалов натуральных чисел.

3. Разработать рекурсивную программу умножения натуральных чисел, используя только операцию сложения.

4. Разработать рекурсивную программу возведения натуральных чисел в натуральную степень, используя только операцию умножения.

5. Рассчитать коэффициенты треугольника Паскаля (бинома Ньютона) рекурсивным способом.

6. Написать рекурсивную программу вычисления n-го члена арифметической прогрессии.

7. Написать рекурсивную программу вычисления суммы n членов арифметической прогрессии.

8. Написать рекурсивную программу вычисления n-го члена геометрической прогрессии.

9. Разработать таблицу простых чисел (в пределах диапазона INTEGER), используя рекурсивный алгоритм («решето») Эратосфена для обнаружения простых чисел среди произвольных натуральных.

10. Написать рекурсивную программу вычисления суммы n членов геометрической прогрессии.

11. Осуществить вычисление наибольшего общего делителя двух натуральных чисел рекуррентным способом.

12. Осуществить вычисление наименьшего общего кратного двух натуральных чисел рекуррентным способом.

13. Создать таблицу «квадратных» чисел, вычислив их рекуррентным способом.

14. Создать таблицу «кубических» чисел, вычислив их рекуррентным способом.

15. Создать таблицу «фигурных» чисел («четырёхугольных пирамидальных»), вычислив их рекуррентным способом.

16. Создать таблицу «фигурных» чисел («пятиугольных пирамидальных»), вычислив их рекуррентным способом.

17. Осуществить рекурсивное вычисление числа перестановок.

18. Осуществить рекурсивное вычисление числа размещений.

19. Осуществить рекурсивное вычисление числа сочетаний.

Часть 2. Рекурсивные логические программы

Разработать рекурсивную программу по материалам программы «Родственные отношения» из 1-й лабораторной работы:

1) предок; 2) предок по мужской линии; 3) предок по женской линии; 4) потомок; 5) потомок по мужской линии; 6) потомок по женской линии; 7) родственник; 8) свойственник; 9) зять (муж кровной родственницы); 10) сват; 11) сватья; 12) невестка (жена кровного родственника).

Часть 3. Приближённые рекурсивные вычисления функций

При выполнении этой части необходимо преобразовать известные разложения в функциональные ряды известных математических функций.

1. Осуществить приближенное рекуррентное вычисление «интегрального синуса», используя разложение в ряд Тейлора-Маклорена.

2. Разработать рекурсивную программу приближенного вычисления функции arctg(x) с помощью разложения в степенной ряд.

3. Рассчитать приближённо функцию «интегральная экспонента» с помощью разложения в степенной ряд.

4. Разработать рекурсивную программу приближенного вычисления функции «интегральный логарифм» с помощью разложения в степенной ряд.

5. Построить приближенную функцию вычисления числа π, используя разложение в степенной ряд функции arctg(x).

6. Вычислить рекурсивно число e, используя разложение в ряд Тейлора-Маклорена.

7. Вычислить приближённо «интегральный косинус», создав соответствующую рекурсивную функцию.

8. Осуществить рекурсивное вычисление «функции ошибок», используя разложение в ряд Тейлора-Маклорена.

Часть 4.

1. Найти «префикс» списка (подсписок, начинающийся с «головы» списка).
2. Найти «суффикс» списка (подсписок с последним элементом в конце).
3. Найти произвольный подсписок списка.
4. Определить первый элемент списка.
5. Определить последний элемент списка.
6. Определить соседние элементы списка.
7. Удалить все вхождения элемента из списка.
8. Вставить заданный элемент в произвольное место списка.
9. Осуществить произвольную перестановку элементов списка.
10. Упорядочить список по возрастанию.
11. Упорядочить список по убыванию.
12. Вставить элемент в заданное место упорядоченного списка.
13. Осуществить разбиение списка на два списка по заданному правилу.
14. Удалить все отрицательные элементы из списка.
15. Продублировать подряд друг за другом все элементы исходного списка.
16. Добавить элемент в «хвост» списка.
17. Осуществить циклический сдвиг элементов в списке.
18. Осуществить удаление всех повторяющихся элементов.

Часть 5.

1. Проверить, является ли список «префиксом» заданного списка.
2. Проверить, является ли список «суффиксом» заданного списка.
3. Проверить, является ли заданный список подсписком другого списка.
4. Проверить, является ли заданный элемент последним элементом списка.
5. Проверить, являются ли два заданных элемента соседними в списке.
6. Проверить, являются ли два заданных списка взаимно обратными.
7. Является ли третий заданный список соединением первого со вторым?
8. Проверить упорядоченность списка по возрастанию.
9. Проверить длину списка на чётность.
10. Состоят ли два заданных списка из компонентов, составляющих третий?
11. Проверить, совпадают ли два списка друг с другом поэлементно.
12. Проверить упорядоченность списка по убыванию.
13. Проверить, является ли заданный список палиндромом.
14. Проверить упорядоченность списка по убыванию.
15. Проверить, является ли количество элементов в списке нечётным.
16. Проверить список на наличие «отрицательных» элементов.
17. Проверить элемент на правильность его положения в списке.
18. Проверить длину списка.

Часть 6.

1. Вычислить сумму элементов числового списка.
2. Вычесть из первого элемента все остальные элементы числового списка.
3. Перемножить элементы числового списка.
4. Поделить первый элемент на все остальные элементы числового списка.
5. Определить n-й элемент списка.
6. Добавить заданное число ко всем элементам числового списка.
7. Вычислить сумму элементов числового списка.
8. Найти среднее арифметическое всех элементов числового списка.
9. Умножить каждый элемент числового списка на заданное число.
10. Найти среднее геометрическое всех элементов числового списка.
11. Поделить каждый элемент числового списка на заданное число.
12. Найти среднее гармоническое всех элементов числового списка.
13. Обратить каждый элемент числового списка.
14. Найти среднее квадратическое всех элементов числового списка.
15. Определить длину списка.
16. Определить, является ли элемент чётным по номеру.
17. Определить, является ли элемент нечётным по номеру.
18. Осуществить нормировку элементов числового списка.

Методические указания

При подготовке к выполнению и сдаче лабораторной работы следует ознакомиться с теоретическими определениями и приёмами рекурсивных вычислений. Рекурсивные программы на языке Visual Prolog 5.2 изучены в [3]. Встроенные алгебраические предикаты подробно рассмотрены в [1-3]. Ознакомиться с теоретическим понятием «список» следует в [3, 5, 9-11]. Также программы обработки списков имеются в [1-3, 5, 9-11]. Встроенные предикаты обработки списков с примерами описаны в [3, 9]. Использование списков в базах данных есть в [11].

Содержание отчёта

1) заголовок (тема) лабораторной работы;
2) цель лабораторной работы;
3) индивидуальные задания на выполнение лабораторной работы;
4) математические формулы или текстовые пояснения к программам;
5) описание предикатов, созданных программистом;
6) рекурсивные программы для числовых вычислений и для обработки списков;
7) результаты работы программ для числовых вычислений и программ для обработки списков;
8) выводы.
9) список использованных источников (литература).

Р а б о т а № 4. **Логические программы, использующие отсечение**

Цель работы − изучение понятия отсечения как процедурного внедрения в декларативный вычислительный процесс. Освоение приёмов использования отсечения в логических программах.

Общие сведения

Все предыдущие лабораторные работы были посвящены изучению основных принципов декларативного логического программирования на Visual Prolog 5.2. Декларативность этих программ состояла в том, что в каждой из них осуществлялась только постановка задачи, но не был показан алгоритм решения. Само решение как ответ на поставленный вопрос автоматически находилось с помощью алгоритма резолюции, встроенного в транслятор этого языка. Эта декларативность логических программ является их главным преимуществом перед программами на алгоритмических языках, реализующих другие парадигмы. Декларативность таких программ позволяет решать весьма непростые задачи. Однако эта же декларативность может служить препятствием тогда, когда необходимо внедриться напрямую в ход вычислительного процесса. Эта необходимость теоретически обоснована тем, что согласно теореме Чёрча исчисление предикатов первого порядка неразрешимо [7]. Возможны ситуации, когда нельзя точно сформировать декларативную постановку задачи, то есть нельзя применить к решению задачи алгоритм резолюции. Также важно, что функционирование алгоритма резолюции, выполняющего поиск решения путём обхода всего дерева возможных решений, порой неэкономно. Вполне реальны ситуации, когда избыточные ходы при поиске решений можно заранее предсказать, а значит, предотвратить. Для таких практических ситуаций существует механизм, называемый ***отсечением*** и реализуемый встроенным предикатом cut, обозначаемым в программах символом восклицательного знака (!). Для выяснения специфики работы отсечения, следует учесть порядок просмотра клауз алгоритмом резолюции. Просмотр осуществляется так:

1) список клауз просматривается по их головам сверху вниз;
2) конъюнкция целей в теле клаузы просматривается слева направо.

Порядок клауз в программах, использующих отсечение, получает принципиально иной смысл. Строгое процедурное определение отсечения выглядит так. «После унификации порождающей цели с головой клаузы, содержащей отсечение, цель выполняется и фиксируется для всех выборов клаузы» [9]. Через отсечение не может быть осуществлён обратный ход при автоматическом поиске решения алгоритмом резолюции.

Процедурное определение отсечения имеет три следствия:

Следствие 1. Отсечение отбрасывает все клаузы с той же головой, что и у клаузы, содержащей отсечение, если эти клаузы расположены после клаузы, содержащей отсечение. Если цель унифицирована с целью в клаузе, содержащей отсечение, и отсечение выполнено, то эта цель не может быть использована при построении решений с помощью клауз с той же головой, расположенных в тексте программы ниже данной.

Следствие 2. Конъюнкция целей, стоящих в теле правила перед отсечением, приводит не более чем к одному решению. Отсечение отбрасывает все альтернативные решения конъюнкции целей, расположенные в теле правила левее отсечения.

Следствие 3. Отсечение никак не влияет на цели, расположенные правее отсечения в клаузе. В случае возвратного движения (отката) алгоритма поиска решения эти цели могут порождать более одного решения. То есть, если отсечение стоит первым в теле клаузы, то клауза функционирует в программе обычным образом. Однако в случае успешного выполнения всех целей этой клаузы, действует следствие 1. Если же отсечение стоит последним в теле клаузы, то все цели данной клаузы могут привести только к одному решению.

Влияние на программу промежуточного положения отсечения в теле клаузы определяется всеми тремя следствиями вместе.

По отношению к результатам работы логической программы принято рассматривать **красное** и **зелёное** отсечения.

Если отсечение нужно для экономии с целью снижения избыточных вычислительных затрат, то отсечение называют *зелёным*. Зелёное отсечение исключает заведомо ложные, ненужные ходы алгоритма резолюции при автоматическом поиске решения. Главный отличительный признак зелёного отсечения: добавление или удаление зелёного отсечения из логической программы не влияет на итоговые результаты работы программы, а влияет лишь на уменьшение или увеличение вычислительных затрат. Зелёное отсечение не меняет декларативный смысл программы.

В то же время *красное* отсечение всегда меняет результат работы логической программы. Можно определить красное отсечение как всякое отсечение, не являющееся зелёным, по принципу исключённого третьего. Любое красное отсечение меняет декларативный смысл программы.

Для демонстрации разных видов отсечений можно рассмотреть ситуацию, реализующую ветвление вычислительного процесса.

Предлагаемое к рассмотрению правило rul(X) печатает фразу «первая ветка», если аргумент правила равен единице, и фразу «вторая ветка», если аргумент правила равен двум. Без использования отсечения это ветвление может быть организовано так:

```
rul(X) :- X=1, write("первая ветка"), nl.
rul(X) :- X=2, write("вторая ветка"), nl.
```

Зелёное отсечение оптимизирует ветвление несколькими способами.

а) Отсечение в конце тела клаузы:

```
rul(X) :- X=1, write("первая ветка"), nl, !.
rul(X) :- X=2, write("вторая ветка"), nl, !.
```

б) Отсечение в середине тела клаузы:

```
rul(X) :- X=1, !, write("первая ветка"), nl.
rul(X) :- X=2, !, write("вторая ветка"), nl.
```

в) Отсечение в начале тела клаузы:

```
rul(1) :- !, write("первая ветка"), nl.
rul(2) :- !, write("вторая ветка"), nl.
```

Важно отметить, что отсечение во вторых (по порядку сверху вниз) правилах никакой процедурной нагрузки не несёт. Достаточно поставить этот предикат только в первое правило, так как после последнего правила rul(X) других правил с такой головой нет. Однако размещение отсечения во все эти правила позволяет сохранить внешний декларативный вид программы, так как в этом случае изменение порядка правил не влияет на результат выполнения программы. Видно, что первые два варианта правила run(X) с отсечениями в конце и в середине клаузы мало отличаются друг от друга, чего нельзя сказать о третьем варианте. При этом во всех трёх случаях отсечение является зелёным, так как удаление отсечения не изменит результат работы программы, а лишь заставит алгоритм поиска решения совершить лишний заход. Приведённый ниже Пример_7 демонстрирует организацию вычисления функции sign (X, Y) с использованием зелёного отсечения. Экономное вычисление нелинейных функций, аналогичных данной, есть типовой случай применения зелёных отсечений.

Пример использования красного отсечения приведён здесь же. Это известное в классической логике исключающее утверждение:

«Мэри любит всех животных, кроме змей».

Такая фраза в логической программе имеет вид:

```
like ("Мэри", X) :- X = «змея», !, fail.
like ("Мэри", _).
```

В программе используется встроенный предикат fail, означающий гарантированный неуспех (fail). Таким образом первое по счёту сверху правило likes ("Мэри", X) никогда не будет считаться в Visual Prolog 5.2 выполненным. Если предикат snake(X) будет успешен, то вслед за ним

автоматически выполнится отсечение - '!', и затем предикат fail сообщит системе о неуспешности этого правила. При этом выполненное отсечение исключит из рассмотрения второе правило. Порядок клауз в этом случае - обязательный, также как и наличие отсечения. Отсутствие здесь отсечения, также как изменение порядка клауз, приведут к изменению результатов работы программы.

Сочетание «отсечение - fail» в программе like (...) является типичным использованием красного отсечения.

Другой пример красного отсечения - остановка рекурсии с помощью отсечения, упраздняющая условие выхода, как, например, при вычислении факториала:

```
factorial (0, 1) :- !.
factorial (N, F) :-
        N1=N-1,
        factorial (N1, F1),
        F=N*F1.
```

Видно, что условия выхода из рекурсии во рекурсивном правиле factorial нет. Это также красное отсечение. Отсутствие отсечения исказит результаты и приведёт к бесконечной рекурсии. Но порядок клауз в этом случае значения не имеет.

Ещё один вариант использования отсечения - оптимизация остатка рекурсии (или оптимизация хвостовой рекурсии). Такая оптимизация используется тогда, когда рекурсивное правило является праворекурсивным (см. работу №3). Тогда отсечение, поставленное перед последним предикатом правила, помогает снизить вычислительные затраты, фактически превращая рекурсивное правило в итерационное (см. так же работу №3). Например, отсечение используется для вставки элемента в заданное место списка: insert_it(Elem, List, List1):

```
insert_it(X, [Y| Ls], [Y|Ls1]) :- asc_order (X,Y), !, insert_it (X, Ls, Ls1).
insert_it(X, Ls, [X| Ls]).
```

Здесь

```
asc_order (X, Y) :- X>Y.
```

Полученная программа вставки элемента в список может быть использована для сортировки списков вставками:

```
sort_insert ([], []).
sort_insert ([X| T], Ls) :- sort_insert (T, Ts), insert_it(X, Ts, Ls).
```

```
/* Пример 7. Использование "отсечения" - предиката cut (!). */
predicates
 signum (real, real)   % предикат для вычисления функции знака числа
 likes (string, string) % предикат "нравится", «любит»
 factorial (integer, integer) % предикат "факториал"
 snake(string)         % предикат "змея"
 cat(string)           % предикат "кошка"
 rat(string)           % предикат "крыса"
 dog(string)           % предикат "собака"
 animal(string)        % предикат "животное"
 p(symbol)             % некий предикат p(...)
 n_p(symbol)           % отрицание предиката p(...) - предикат n_p(...)
clauses
    /* Вычисление функции знака числа X - пример зелёного отсечения. */
signum (X, 1) :- X>0, !.
signum (X, 0) :- X=0, !. % Порядок клауз sign (.,.) - произвольный.
signum (X, -1) :- X<0.

  %Пример красного отсечения: "Мэри любит всех животных, кроме змей".
likes ("Мэри", X) :- snake(X), !, fail.
likes ("Мэри", X) :- animal(X).
                     % Обязательный порядок у правил likes.
animal(X) :- dog(X);
             rat(X);
             cat(X);
             snake(X).

dog("Дружок").
rat("Чучундра").
snake("Нагайна").

cat("Мурка").
cat("Барсик").

   /* Пример использования красного отсечения для выхода из рекурсии. */

factorial (0, 1) :- !.          % Условие выхода: 0! = 1.
factorial (N, F) :- N1=N-1, factorial (N1, F1), F=N*F1. % Рекурсивное правило.
            /* Последовательность клауз factorial (.,.) - обязательная. */

% Пример использования красного отсечения - предикат отрицания n_p.

p(var).                 % Аргумент предиката p(X) - некая величина var.

n_p(X) :- p(X), !, fail.    % Если успешен p(X), то неуспешен n_p(X).
n_p(X) :- true.             % Если p(X) неуспешен, то успешен n_p(X).
            /* Порядок клауз в правилах n_p обязательный*/
```

Использует отсечение и праворекурсивное правило удаления отрицательных элементов из списка:

```
discard_negativ ([], []).
discard_negativ ([H| T], Result) :- H<0, !, discard_negativ(T, Result).
discard_negativ ([H| T], [H| Result) :- discard_negativ(T, Result).
```

С помощью отсечения удобно иллюстрировать функционирование встроенного предиката второго порядка not(...). Этот предикат возвращает значение fail («неуспех»), когда предикат, являющийся его аргументом, успешен (возвращает значение true). И, наоборот, предикат not(...) возвращает значение true, когда аргумент предиката неуспешен (возвращает значение fail). Отсечение помогает понять процедуру его функционирования:

```
not(P) :- P, !, fail.
not(P).
```

или

```
not(P) :- P, !, fail; true.
```

Но написать такую программу пользователь не может, так как в исчислении предикатов 1-го порядка **предикат не может быть аргументом предиката**. Пример_7 показывает, как создать частный предикат n_p, противоположный данному предикату p, вместо универсального встроенного предиката not.

Задания для лабораторной работы №4

Часть 1

Запрограммировать с использованием «зелёного» отсечения нелинейности:

1) «реле»;
2) «зона нечувствительности»;
3) «единичный скачок»;
4) функция «прямоугольник» - rect(x);
5) функция Радемахера;
6) функция Хэвисайда.
7) функция «максимум из двух переменных».
8) функция «минимум из двух переменных».
9) «модуль с насыщением и зоной нечувствительности»;
10) «модуль с зоной нечувствительности»;
11) «реле с зоной нечувствительности»;
12) «модуль»;
13) «насыщение»;
14) «зона нечувствительности с насыщением»;
15) «модуль с насыщением».

Часть 2

Запрограммировать с помощью «красного» отсечения исключающие утверждения:

1. «Все игры с мячом - командные, кроме художественной гимнастики».
2. «Даша любит все цвета, кроме чёрного».
3. «Всё покупается, кроме любви».
4. «Юле нравятся любые юбки, кроме мини».
5. «Миша любит все ягоды, кроме горьких».
6. «Света ест любую пищу, кроме мучной и жирной».
7. «Любую потерю можно вернуть, кроме времени и здоровья».
8. «Петя ест любую еду, кроме мясной».
9. «Вера не любит игровые виды спорта, а остальные виды спорта любит».
10. «Игорь любит все уроки, кроме физкультуры».
11. «Богородица молится за всех, кроме царя Ирода».
12. «Нет никого, кроме Бога, без греха».
13. «Прощаются все грехи, кроме греха Иуды».
14. «Хороши все жанры в искусстве, кроме скучного».
15. «Все грибы, кроме поганок, съедобны».
16. «Все рыбы, кроме акул, добры к детям».
17. «Маша любит все фрукты, кроме кислых».
18. «Виктор любит все виды спорта, кроме шахмат и шашек».

Часть 3.

Модифицируйте рекурсивные программы из 3-й лабораторной работы, изменив в них условия выхода с использованием «красного» отсечения.

Часть 4.

Исследуйте возможности оптимизации хвостовой рекурсии с использованием отсечения в праворекурсивных программах из 3-й работы.

Методические указания

Понятие «зелёного» и «красного» отсечения подробно рассмотрено в [3, 10]. Характерные примеры отсечений есть в [9].

Содержание отчёта.

1) тема лабораторной работы;
2) цель лабораторной работы;
3) индивидуальное задание;
4) тексты логических программ, содержащие отсечение;
5) результаты работы программ и выводы;
6) тексты логических программ, содержащие отсечение;
7) результаты работы программ и выводы.

Р а б о т а № 5. **Динамические предикаты внутренних баз фактов**

Цель работы - изучение приёмов логического программирования с использованием динамических предикатов внутренних баз фактов языка Visual Prolog 5.2. Безымянная и именованные внутренние базы фактов. Изучение встроенных предикатов языка Visual Prolog 5.2, предназначенных для работы с динамическими предикатами внутренних баз фактов.

Общие сведения

Настоящая лабораторная работа посвящена изучению принципиально иных методов логического программирования по сравнению с приёмами программирования, которые освоены при выполнении предыдущих лабораторных работ.

Декларативный принцип, на котором основано логическое программирование, помимо достоинств обладает и недостатками. Вычисленные с помощью какого-либо правила значения переменных сохраняются только в пределах этого правила и утрачиваются при переходе механизма поиска решения к другому правилу. Сохранить вычисленные в правиле значения оказывается возможным, только напечатав их в окне консоли с помощью предикатов вывода. Лишь факты хранят доступные для всей программы константные значения переменных в течение всего времени работы программы. Но факты, заданные при создании программы, не могут измениться в процессе функционирования программы.

Для преодоления таких затруднений язык Visual Prolog 5.2 имеют возможности фиксировать или изменять вычисленные значения во время выполнения программы путём добавления или удаления динамических фактов. Добавляемые и удаляемые факты изменяют внутренние базы фактов программы. Предикаты для таких фактов называют динамическими предикатами внутренней базы фактов. Эти базы фактов называют *внутренними* потому, что Visual Prolog 5.2 имеет возможность взаимодействовать и с внешними базами данных. В языке Visual Prolog 5.2 динамические предикаты внутренних баз фактов должны быть предварительно описаны, как и обычные предикаты, но только в специальных разделах facts, так как добавлять и удалять в логическую программу можно только факты, а правила нельзя. Структура программы на Visual Prolog 5.2 с использованием динамических предикатов внутренних баз фактов имеет следующий вид:

domains

<описание пользовательских типов данных,>

<простых и структурированных>

facts

<описание динамических предикатов ***безымянной*** *внутренней базы фактов>*

facts - *<имя 1-й базы фактов>*

<описание динамических предикатов 1-й внутренней базы фактов с именем>

…

facts - *<имя n-й базы фактов>*

<описание динамических предикатов n-й внутренней базы фактов с именем>

predicates

<описание «обычных», статических предикатов>

clauses

<факты и правила логической программы>

Программа может иметь только одну безымянную базу фактов, но несколько баз фактов, имеющих имена. Взаимное расположение разделов, объявляющих предикаты, произвольно. Имена динамических баз фактов, как и имена предикатов, тоже могут быть произвольными. Только необходимо, чтобы они начинались со строчной латинской буквы (db1, d_p2 и т. п.).

Наличие нескольких баз фактов в одной программе не является синтаксически необходимым и определяется пользователем в зависимости от желания распределить различные динамические факты по разным разделам.

Для работы с безымянной базой фактов в языке Visual Prolog имеются пять встроенных предикатов второго порядка:

assert (Факт) - добавить Факт в произвольное место безымянной базы фактов;

asserta (Факт) - добавить Факт в начало безымянной базы фактов;

assertz (Факт) - добавить Факт в конец безымянной базы фактов;

retract (Факт) - удалить одно вхождение Факта из безымянной базы фактов;

retractall (Факт) - удалить все вхождения Факта из безымянной базы фактов.

В дополнение к этим предикатам второго порядка имеются два встроенных служебных предиката. Один из них позволяет сохранять факты, накопленные в безымянной динамической базе фактов, в файле с заданным именем:

save (Имя_Файла).

Другой встроенный служебный предикат позволяет добавлять факты в безымянную базу фактов из заданного файла, в котором эти факты были ранее каким-либо образом накоплены:

consult (Имя_Файла).

Схожие предикаты имеются и для работы с именованными базами фактов. Вместо одного аргумента эти предикаты имеют два аргумента, так как второй аргумент содержит имя внутренней базы фактов. Предикаты для именованных динамических баз данных таковы:

assert (Факт, Имя_базы_ фактов) - добавить Факт в произвольное место внутренней базы фактов, имеющей Имя_базы_фактов;

asserta (Факт, Имя_базы_ фактов) - добавить Факт в начало внутренней базы фактов, имеющей Имя_базы_фактов;

assertz (Факт, Имя_базы_ фактов) - добавить Факт в конец внутренней базы фактов, имеющей Имя_базы_фактов;

retract (Факт, Имя_базы_ фактов) - удалить одно вхождение Факта из внутренней базы фактов, имеющей Имя_базы_фактов;

retractall (Факт, Имя_базы_ фактов) - удалить все вхождения клаузы Факт из динамической базы данных, имеющей Имя_базы_фактов;

save (Имя_Файла, Имя_базы_ фактов) - сохранить в файле, имеющем Имя_Файла, факты, накопленные во внутренней базе фактов, имеющей Имя_базы_ фактов;

consult (Имя_Файла, Имя_базы_фактов) - добавить все факты из файла, имеющего Имя_Файла, во внутреннюю базу фактов, имеющую Имя_базы_ фактов.

Предикаты, используемые в именованной внутренней базе фактов, должны быть объявлены в соответствующем разделе логической программы facts (допускается и название database). При поиске решения алгоритм резолюции работает со всеми предикатами логической программы, не делая различия между предикатами, объявленными в разделе predicates, и предикатами, объявленными в одном из разделов facts. Использование любого из предикатов consult фактически изменяет набор фактов логической программы, что напоминает использование предикатов группы assert. Использование нескольких именованных внутренних баз фактов бывает необходимым только тогда, когда требуется сохранять различные наборы накопленных фактов в различных файлах. Далее приведены несколько примеров, содержащих фрагменты логических программ, в которых использованы динамические предикаты внутренних баз фактов.

1. Программа с повторами, осуществляющая создание внутренних баз фактов путём копирования всех фактов с предикатом p (X, Y) в факты с предикатом dp (X, Y) из безымянной внутренней базы фактов:

```
assert_facts :- p (X, Y), assertz (dp (X, Y)), fail.
assert_ facts :- !.
```

Наличие предиката fail заставляет механизм резолюции продолжать поиск решения, пока не будут просмотрены все факты типа p (X, Y). Успешный выход из повторов после просмотра всех заданных фактов осуществляется с помощью отсечения.

2. Сохранение безымянной внутренней базы фактов, накопленной в файле с именем dbs_file.txt непосредственно из раздела goal:

goal: assert_facts; save(“dbs_file.txt”).

3. Программа с повторами, осуществляющая удаление всех фактов вида dp (X, Y) из динамической базы данных:

```
clear_ facts :- retract(dp(_,_)), fail.
clear_ facts :- !.
```

Механизм функционирования программы clear_facts аналогичен механизму программы assert_facts. Для удаления здесь используется недетерминированный предикат однократного удаления retract(...), используемый многократно. Возможно такое же по результату однократное использование предиката retractall(...), например: retractall (dp (_, _)).
Возможны и другие случаи использования retractall.

4. Подключение фактов из файла dbs_file.txt к внутренней базе фактов db1 непосредственно из раздела goal:

goal consult (“dbs_file.txt”, db1).

Рассмотренные примеры показывают разнообразие возможностей использования динамических предикатов внутренних баз фактов.

Задания для лабораторной работы № 5

Используя материалы 1-й, 2-й и 3-й лабораторных работ в соответствии с вариантами заданий к ним, выполнить следующее:

а) создать внутреннюю динамическую базу данных;

б) сохранить созданную внутреннюю базу данных в файле;

в) изучить текст созданного в пункте б) файла;

г) подключить факты, хранимые в файле, к новой логической программе.

Это нужно выполнить как с неструктурированной информацией из лабораторной работы №1, так и со структурно организованной информацией из лабораторной работы №2, аналогично примерам 1 - 4 настоящей работы.

Методические указания

«Самопрограммирование» с помощью предикатов assert и retract изучено в [10], подробнее динамические базы данных рассмотрены в [9], полный набор встроенных предикатов для работы с внутренними базами фактов языка Visual Prolog 5.2 имеется в [3].

Содержание отчёта:

1) тема лабораторной работы;
2) цель лабораторной работы;
3) индивидуальное задание;
4) тексты логических программ с динамическими предикатами внутренних баз данных;
5) результаты работы логических программ;
6) выводы;
7) список использованных источников.

Р а б о т а № 6. **Оптимизация логических программ на Visual Prolog 5.2**

Цель работы - знакомство с разными способами оптимизации логических программ: переход от левой к правой рекурсии, изучение различий между итерационными и рекурсивными программами, переход от рекурсивной к итерационной программе, использование переменных-накопителей, оптимизация хвостовой рекурсии, использование динамических предикатов внутренних баз фактов, типовые металогические предикаты.

Общие сведения

Рекурсивные логические программы - простой и весьма эффективный инструмент в решении широкого круга практических задач. Структура рекурсивной программы часто представляется очевидной уже при грамотном формулировании поставленной задачи (см. работу №3). Однако рекурсивные вычислительные процедуры порой обладают и недостатками.

Быстрое заполнение стека и экспоненциальный рост объёма вычислений – известные проблемы в практике использования рекурсии. При вызове рекурсивной процедуры происходит последовательное пошаговое движение от точки вызова к клаузе, означающей окончание рекурсии. При каждом шаге в стеке резервируется память под промежуточные значения переменных, используемых в рекурсивном вычислении. После этого осуществляется обратное движение к точке вызова, при котором заполняются значениями все зарезервированные области памяти.

Такое возвратно-поступательное движение требует существенных затрат оперативной памяти и вычислительных затрат, объём которых растёт на порядок быстрее, чем число шагов в аналогичном циклическом вычислении.

Однако возможно преобразовать эти процедуры так, чтобы исключить возвратно-поступательный ход рекурсивного процесса и преобразовать его в линейный, односторонне направленный. Такие логические программы называют **итерационными**. Это название программы получили в связи со своим фактическим сходством с итерационными функциями в императивных языках программирования. В отличие от ранее рассмотренных рекурсивных программ в итерационных программах требуемая оперативная память и вычислительные затраты зависят от числа шагов линейно, а не экспоненциально.

Итерационные программы также являются рекурсивными, но со своими отличительными признаками. Основным признаком итерационной программы на практике является праворекурсивный характер рекурсивного правила. То есть предикат рекурсивного вызова должен быть крайним правым в тексте правила при записи его в одну строку, то есть последним в списке целей. Таково правило «предок» в последнем примере к работе №3. Являясь праворекурсивным, оно более экономично, чем леворекурсивный аналог. В варианте «прямой_предок_по_мужской_линии» оно будет выглядеть как [3]:

parent (X, Y) :- father (X,Y).

parent (X, Y) :- father (X, Z), parent (Z,Y).

Причины преимущества именно таких программ можно понять, если перейти от декларативного смысла логических программ к их процедурному смыслу. Так как просмотр клауз при поиске решения ПРОЛОГ-машиной осуществляется сверху вниз, а просмотр целей внутри каждой клаузы слева направо, то рекурсивное обращение на последнем шаге поиска решения может избавить ПРОЛОГ-машину от необходимости запоминать промежуточные значения для возвратного движения рекурсии.

Не всякие рекурсивные программы могут быть легко представлены в праворекурсивном виде как программа «предок». В этом случае нужно включать в итерационные программы дополнительные переменные, называемые **накопители**. Благодаря накопителям основные аргументы предикатов не изменяются внутри клаузы на каждом шаге рекурсии. Находясь в голове и в теле клаузы, основные аргументы предикатов лишь стоят на фиксированных позициях, никак иначе не участвуя в вычислениях.

Накопители, в соответствии со своим наименованием, «накапливают» значения также как накапливают результирующие значения вычислений и значения счётчиков циклов рабочие переменные в императивных языках. Основные аргументы предикатов в итерационных вычислениях активны только в моменты начала и конца итерации. Накопленные за время работы рекурсивного правила значения переходят от накопителей к основным аргументам в точке останова рекурсии (в точке конца итерации), когда имена основных аргументов и соответствующих им накопителей совпадают. При этом старт рекурсии задаёт конкретные начальные значения накопителей, в то время как основные аргументы остаются неозначены.

Старт итерации удобно оформить в виде правила, где голова - предикат только с основными аргументами, а тело клаузы - аналогичный предикат, но имеющий наряду с основными и дополнительные аргументы-накопители (см. Пример 5). Само наличие двух точек: точки начала и точки конца итерации (в отличие от одной точки остановки рекурсии) есть отличительная черта итерационных программ с накопителями, формально отражающая одностороннее направление движения вычислительного процесса.

Введение переменных-накопителей в рекурсивные клаузы оказывается необходимым, потому что значения переменных сохраняются только внутри клаузы при обращении к ней и не сохраняются при переходе к другой клаузе.

Рассмотрим переход к итерационным процедурам при вычислении чисел Фибоначчи. В работе №5 было приведён пример классического дважды рекурсивного вычисления:

```
fib (1, 1).
fib (2, 1).
fib (N, Fib) :-
        N>2,
        N1=N-1,
        N2=N-2,
        fib (N1, Fib1),
        fib (N2, Fib2),
        Fib=Fib1+Fib2.
```

Такое вычисление порождает ветвящееся дерево целей на промежуточных этапах вычислений и приводит к лавинообразному росту объёмов требуемой памяти и вычислительных затрат.

С целью уменьшения этих затрат двойная рекурсия преобразуется в обычную «однократную» рекурсию с помощью введения дополнительного, третьего аргумента в предикат fib:

```
fib (1, 1, 0). // точка остановки рекурсии

fib (N, Fib, Fib1) :-
                    N>1, // условие выхода
                    N1=N-1, // номер предыдущего шага
                    fib (N1, Fib1, Fib2), // рекурсивный вызов
                    Fib=Fib1+Fib2. //очередное вычисление числа
```

Следующий этап - создание итерационной (праворекурсивной) программы.

Аналогичное предложенной процедуре вычисление чисел Фибоначчи на Visual Prolog 5.2 возможно программой с одним накопителем:

```
fib_iter (N, Fib) :- fib(N, 1, 0, Fib). // начало итерации

fib (N, F1, F2, Fib) :-
                    N>1,
                    N1= N-1,
                    NewF1= F1 + F2,
                    fib (N1, NewF1, F1, Fib). // "правая" рекурсия

fib (1, Fib, _, Fib). // передача информации накопителю
                    // (конец итерации)
```

Таким образом полученное рекурсивное правило имеет предикат головы клаузы последним (правым) в теле правила.

В примере 8 приведены итерационные аналоги рекурсивной программы вычисления факториала натурального числа с использованием различного количества накопителей. Итерационные программы достаточно сложны, в связи с чем особенно важно выполнение заданий настоящей лабораторной работы. Преобразование рекурсивных правил к итерационному виду не является последним этапом на пути оптимизации рекурсивных программ. Дальнейшая оптимизация в ряде случаев возможна также при использовании отсечения, рассмотренного ранее в работе № 4 специального приёма, позволяющего изменять процедурный смысл логической программы.

```
% Пример 8. Переход от рекурсивных к итерационным программам.

predicates
 nondeterm fact (integer, integer)
 nondeterm fact_it (integer, integer)
 nondeterm fact_it (integer, integer, integer, integer)
 nondeterm fact_iter (integer, integer)
 nondeterm fact_iter (integer, integer, integer)

clauses

/* Рекурсивное вычисление факториала по формуле: N! = (N-1)! * N.    */

        fact (0, 1).
        fact (1, 1).
        fact (N, F) :-
                N>1,
                N1=N-1,
                fact (N1, F1),
                F=F1*N.

/* Итерационное вычисление факториала */

        fact_it (N, F) :- fact_it (0,N,1,F).
        fact_it (I, N, T, F) :-
                                I<N,
                                I1=I+1,
                                T1=T*I1,
                                fact_it (I1, N, T1, F).
        fact_it (N, N, F, F).

 % Упрощённое итерационное вычисление факториала N! = N*(N-1)*...*2*1.

        fact_iter (N, F) :- fact_iter(N,1,F).
        fact_iter (N, T, F) :-
                                N>0,
                                T1=T*N,
                                N1=N-1,
                                fact_iter (N1, T1, F).
        fact_iter (0, F, F).
```

Важно видеть, что Visual Prolog 5.2 допускает использование в одной программе нескольких предикатов с одним именем, но разным количеством аргументов.

Переход от рекурсии к итерации возможен не только в числовых вычислениях, но и при обработке списков. Переход от левой к правой рекурсии с помощью накопителей можно видеть на примере программы обращения списков reverse. В лабораторной работе №4 показана программа обращения reverse, которая представляет собой левую рекурсию и вложенную рекурсию, использующую программу соединения списков append. Эта программа обращения списков из лабораторной работы №4 эффективна, но не оптимальна с позиции вычислительных затрат.

Обращение списка может быть представлено как правая рекурсия без вложенной рекурсии, если добавить третью переменную-накопитель Acc (accumulator). При этом вызов функции reverse с тремя аргументами может быть организован с помощью традиционной функции reverse с двумя аргументами: входным списком List и выходным обращённым списком Result:

```
reverse (List, Result) :- reverse (List, [], Result).  % начало вычислений

reverse ([Head| Tail], Acc, List) :- reverse (Tail, [Head| Acc], List).
reverse ([], List, List).                    % конец вычислений
```

Из программы видно, как при пошаговом вызове рекурсивного правила из входного списка отделяется голова и в обратном порядке помещается в список-накопитель (список-аккумулятор Acc) до тех пор, пока не будет исчерпан входной список. Так как в начале вычислений список-накопитель пуст, в конце вычислений список-накопитель будет заполнен элементами входного списка в обратном порядке. В этот момент содержание списка-накопителя передаётся выходному списку. Вычислительный процесс в этом случае представляет собой не возвратно-поступательный процесс, а процесс линейный и односторонне направленный.

Названные примеры оптимизации вычислительных затрат с помощью перехода от левой рекурсии к правой рекурсии (от рекурсии к итерации) как путём непосредственной перестановки целей в рекурсивных правилах, так и путём введения дополнительных аргументов в предикатах не исчерпывают возможности по оптимизации рекурсивных программ. Помимо исключения многократных рекурсий (см. программу, вычисляющую fib (N, Fib, Fib1)), а также исключения вложенных рекурсий (см. пример reverse (List, [], Result)), возможна дальнейшая оптимизация рекурсивных программ с помощью отсечения, рассмотренного в лабораторной работе № 4.

Строго говоря, как хорошо видно из материала лабораторной работы №4, любое применение отсечения так или иначе оптимизирует программу. Если это не сокращение вычислительных затрат, то это некоторое сокращение

объёма кода. Однако отсечение у точки останова рекурсии, представленное в лабораторной работе № 4, позволяет исключить условие выхода из рекурсии, что заметно влияет на экономию времени выполнения программы. Также весьма заметно, но на порядок больше, экономятся вычислительные затраты при оптимизации «хвостовой» рекурсии (остатка рекурсии).

Оптимизация «хвостовой» рекурсии [9] (или же остатка рекурсии [3]) выполняется по схеме:

$$A :- B_1, B_2, \ldots, B_n, !, A.$$

Здесь A и B_i – предикаты, цели рекурсивного правила.

Отсечение «хвоста» рекурсии позволяет сколь угодно раз находить решения для «хвоста» A правее отсечения, но разрешает только одно решение для каждой цели B_i из конъюнкции целей B_1, B_2, …, B_n левее отсечения. Эффект сокращения затрат происходит по этой причине.

Эффективность такой оптимизации в каждом отдельном случае следует экспериментально проверять. Предполагается, что это «зелёное» отсечение, но теоретически не доказано, что это всегда так. На практике известны редкие примеры, когда такое отсечение оказывается «красным». Очевидно, что это эмпирический, но при этом весьма эффективный приём логического программирования.

Следует проанализировать рекурсивные правила и, если они являются праворекурсивными, оптимизировать их с помощью отсечения «хвоста» рекурсии, проверив правильность итоговых результатов.

Таким образом можно оптимизировать праворекурсивное правило, полученное выше при переходе к правой рекурсии:

fact_iter (N, T, F) :- N>0, T1=T*N, N1=N-1, **!**, **fact_iter** (N1, T1, F).

Экономия кода в рекурсивном правиле и отсечение хвостовой рекурсии можно видеть в программах из лабораторной работы № 4. В правиле insert_it и в правиле discard_negativ, предикат cut (!) отсекает остаток рекурсии и не позволяет перейти к клаузе с такой же головой, стоящей ниже.

Использовать оптимизацию «хвостовой» рекурсии имеет смысл только тогда, когда левее рекурсивного вызова в теле правила стоят другие цели. Если в теле правила, кроме рекурсивного вызова, ничего нет, то не следует отсекать рекурсивный вызов, так как экономии вычислительных затрат не будет. По этой причине бессмысленно использовать отсечение в таких рекурсивных программах обработки списков, как append, select, member.

В итоге можно собрать все рассмотренные в работе методы оптимизации рекурсивных программ в общий список.

Для оптимизации рекурсивных программ следует:

а) устранить многократные рекурсии;

б) устранить вложенные рекурсии;

в) переделать левые рекурсии в правые;

г) отсечь «хвост» рекурсии;

д) отсечь точку останова, устранив условие выхода.

При этом порядок этих оптимизаций не является обязательным. Все пункты могут быть выполнены в совокупности, но каждый отдельный пункт может и не выполняться, если нет соответствующих синтаксических предпосылок. Программист сам может выбрать, какие именно методы он применит. Важно лишь заметить, что все пять перечисленных методов не противоречат друг другу, а дополняют друг друга. Также важно заметить, что многократные рекурсии не обязательно бывают затратными. Например, если двукратная рекурсия «числа Фибоначчи» чрезвычайно затратна, то двукратная рекурсия «Ханойская башня» [3] оптимальна с позиции вычислительных затрат.

Предложенные пять способов оптимизации рекурсий не исчерпывают всех способов оптимизаций рекурсивных вычислений. Ещё один способ стоит особняком, так как может применяться не совместно со всеми пятью вышеназванными способами оптимизации, а вместо всех этих пяти способов. Это оптимизация, использующая динамические предикаты встроенных баз фактов языка Visual Prolog 5.2, изученные в лабораторной работе № 5.

Так, в приведённом ниже Примере 9 использование динамических предикатов сокращает до минимума требуемый объём стека (оперативной памяти) и вычислительные затраты при вычислении чисел Фибоначчи с помощью двукратной рекурсии. Двукратная рекурсии влечёт повторяющиеся вычисления. Первое и второе числа Фибоначчи равны 1. Вычисления 7-го числа Фибоначчи провоцирует ветвящийся порядок вычислений:

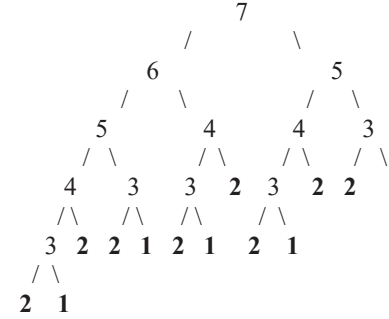


Схема разветвляющегося вычислительного процесса до оптимизации

Использование динамических предикатов внутренних баз фактов позволяет устранить повторы в вычислениях, сохраняя вычисленные числа с помощью динамических фактов [1, 2, 10]. Программа в Примере 9 работает как самопрограммирующаяся информационная система.

```
% Пример 9. Применение динамических предикатов внутренних баз фактов
facts
  fibd (integer, integer)
predicates
  fib (integer, integer) % Предикат для вычисления чисел Фибоначчи
clauses
fib (N, F) :- fibd(N, F),!. %Извлечение информации из внутренней базы фактов
fib (1, 1) :- asserta(fibd(1,1)),!.
fib (2, 1) :- assertz(fibd(2,1)),!.
fib (N, F):-
            N1=N-1,
            fib (N1, F1),
            N2=N-2,
            fib (N2, F2),
            F=F1+F2,
           assert (fibd (N, F)). % Добавление нового факта во внутреннюю базу

goal: fibd (N, Fib). % Запрос информации из динамической базы данных
No Solution
goal: fib (8, Fib). % Вычисление 5-го числа Фибоначчи.
Fib=21
1 Solution
goal: fibd (N, Fib). % Повторный запрос из динамической базы данных.
N=1, Fib=1
N=2, Fib=1
N=3, Fib=2
N=4, Fib=3
N=5, Fib=5
N=6, Fib=8
N=7, Fib=13
N=8, Fib=21
8 Solution
```

Из примера видно, что накапливающиеся факты превращают ветвящиеся вычисления в линейный вычислительный процесс:

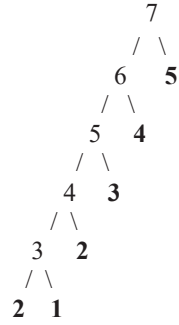


Схема линейного вычислительного процесса после оптимизации

Динамические предикаты позволяют выполнить другую оптимизацию, оптимизацию при повторах ответов на вопросы (запросы).

Из-за того, что алгоритм автоматического поиска решений (алгоритм резолюции) фиксирует пути решений, но не фиксирует сами решения, возможны повторы в ответах на вопросы (запросы), когда одни и те же решения получены разными путями. Чтобы избежать этого, необходимо запоминать полученные при работе программы ответы как динамические факты. Вот, например, вопрос (запрос) к базе данных из работы №1, состоящей из фактов типа father (X, Y). Использование внутренних баз фактов и предиката not исключает паразитные повторы ответов [1-2, 12]:

```
goal
        father (X, _),          % Обращение к фактам father (X, _).
        not (d_father (X)), % Есть ли соответствующий факт в базе фактов?
        assert (d_father (X)), % Добавить, если такого факта не существует.
        write(X),               % Распечатать значение аргумента предиката.
        fail; true.             % Продолжить просмотр.
```

Комментарии показывают то, как в этой программе функционирует механизм внутренних баз фактов. Фактически таким способом можно осуществлять накопление информации, полученной в процессе работы программы. В императивных языках программирования эти действия выполняются с помощью оператора присваивания, отсутствующего в логическом программировании.

Сокращение наименований предикатов, используемых в программе

Использование одних и тех же предикатов в разных смыслах, использование их аргументов как входных, так и выходных, также можно отнести к некоторому виду оптимизации программы. Для этого в языке Visual Prolog 5.2 используются так называемые **типовые металогические предикаты** [1-3].

Дальнейшее изложение будет посвящено изучению особенностей использования встроенных предикатов языка Visual Prolog 5.2, называемых **bound** и **free**, предназначенных для определения типов аргументов предикатов с точки зрения того, входные они (i) или выходные (o), означены они или нет.

Если аргументы означены при обращении к предикату, то есть принимают конкретные заданные значения, то такие аргументы называют связанными: **bound**. Выходные аргументы при обращении к предикату неозначены, то есть являются свободными переменными: **free**. В момент успешного выполнения предиката, когда предикат возвращает true, выходные аргументы принимают вычисленные значения. Предикаты могут иметь как входные, так и выходные аргументы. Часто одни и те же аргументы предикатов выступают как в роли входных, так и в роли выходных аргументов. К предикату «X является отцом Y»: father (X, Y) можно обратиться семью способами:

1) father (X, Y).
   o o "Кто кому приходится отцом?"
2) father ("Иван", Y).
   i o "Кому приходится отцом Иван?"
3) father (X, "Мария").
   o i "Кто приходится отцом Марии?"
4) father ("Иван", " Мария").
   i i "Иван - отец Марии?"
5) father ("Иван", _).
   i i "Является ли Иван кому-либо отцом?"
6) father (_, "Иван").
   i i "Указан ли отец Иван в нашей программе?"
7) father (_, _).
   i i "Есть ли информация об отцах в нашей программе?"

Аргументы предиката выступают как в роли входных, так и в роли выходных. Такие предикаты можно условно назвать универсальными. Вместе с этими универсальными предикатами существуют предикаты со строго заданными с позиции связности типами аргументов. В этих предикатах аргументы могут быть только входными или только выходными. Таковы, например, программа вычисления суммы элементов списка:

```
sum (List, Sum),
      i     o
```

или программа вычисления чисел Фибоначчи:

```
fib (N, Fib).
   i   o
```

Порой необходимо определить тип означенности аргументов. Определение означенности происходит с помощью типовых металогических предикатов, которыми в языке Visual Prolog 5.2 являются предикаты:
free(V) - выполнен, если переменная V неозначена (свободна);
bound(V) - выполнен, если переменная V означена (связана).

В Примере 10 представлены программы с использованием этих предикатов. Созданы универсальные предикаты plus (A, B, C) и mult (A, B, C), переделав узкоспециальные программы:

```
plus (A, B, C) :- C=A+B.
      i  i  o
mult (A, B, C) :- C=A*B.
      i  i  o
```

```
/* Пример_10. Использование предикатов bound и free [1-2, 10] */
predicates
 nondeterm plus (real, real, real)
 nondeterm mult (real, real, real)
clauses
 plus (A, B, C) :- bound(A), bound(B), C=A+B.
 plus (A, B, C) :- bound(A), bound(C), B=C-A.
 plus (A, B, C) :- bound(B), bound(C), A=C-B.

 mult (A, B, C) :- bound(A), bound(B), C=A*B.
 mult (A, B, C) :- bound(A), bound(C), A<>0, B=C/A.
 mult (A, B, C):- bound(B), bound(C), B<>0, A=C/B.
 mult (A, _, C) :- bound(A), bound(C), A=0, write("Деление на ноль!").
 mult (_, B, C) :- bound(B), bound(C), B=0, write("Деление на ноль!").
Goal: plus (4, 3, C).
C=7
Goal: plus (4, B, 7).
B=3
Goal: plus (A, 3, 7).
A=4
Goal: plus (4, 3, 7).
Yes
Goal: mult (12, 5, 60).
Yes
Goal: mult (A, 5, 60).
A=12
Goal: mult (12, B, 60).
B=5
Goal: mult (12, 5, C).
C=60
```

Задания для лабораторной работы № 6

Часть 1

1. Разработать праворекурсивную программу «алгоритм Евклида для поиска наибольшего общего делителя двух натуральных чисел».
2. Создать итерационную программу вычисления n-го члена арифметической прогрессии.
3. Разработать итерационную программу возведения натуральных чисел в натуральную степень, используя только операцию умножения.
4. Написать итерационную программу вычисления суммы n первых членов арифметической прогрессии.
5. Написать итерационную логическую программу вычисления n-го члена геометрической прогрессии.
6. Написать итерационную программу вычисления суммы первых n членов геометрической прогрессии.
7. Создать таблицу «треугольных» чисел, вычислив эти числа итерационным способом.

Часть 2

Исследовать право и леворекурсивные программы:

1) «потомок»,
2) «предок по прямой мужской линии»,
3) «предок по прямой женской линии»,
4) «потомок по прямой мужской линии»,
5) «потомок по прямой женской линии».

Часть 3

Переделать программы заданий работы №3 из рекурсивных в итерационные.

Часть 4

Используя динамические предикаты внутренних баз фактов и текст лабораторной работы №1, создать запросы, ответы на которые не будут иметь паразитных повторов.

Часть 5

На основе материалов лабораторных работ № 3 и № 5 в соответствии с вариантами задания на эти работы оптимизировать рекурсивные программы. Для этого использовать встроенные предикаты внутренних динамических баз фактов.

Часть 6

Создать универсальный предикат, выполняющий следующие действия:

1) вычисление синуса (арксинуса) аргумента;
2) вычисление степени (арифметического корня, логарифма);
3) вычисление косинуса (арккосинуса) аргумента;

4) вычисление n-го члена арифметической прогрессии (разности, номера, начального члена);
5) вычисление тангенса (арктангенса) аргумента;
6) вычисление n-го члена геометрической прогрессии (частного, номера, начального члена);
7) вычисление котангенса (арккотангенса) аргумента;
8) вычисление суммы n членов арифметической прогрессии (разности, номера, начального члена);
9) вычисление косеканса (арккосеканса) аргумента;
10) вычисление суммы n членов геометрической прогрессии (частного, номера, начального члена);
11) вычисление квадратического значения двух чисел (одного из составляющих квадратическое значение).
12) вычисление бесконечной суммы геометрической прогрессии (частного, начального члена);
13) вычисление среднего арифметического двух чисел (одного слагаемого);
14) вычисление результата целочисленного деления с остатком (делимого, делителя);
15) вычисление среднего квадратического двух чисел (одной из квадратических составляющих);
16) вычисление среднего геометрического двух чисел (одного сомножителя);
17) вычисление среднего гармонического двух чисел (одной из гармонических составляющих);
18) вычисление секанса (арксеканса) аргумента;

Методические указания

При выполнении заданий к настоящей лабораторной работе предлагается сначала создать рекурсивную программу, а затем осуществить переход от рекурсивных к итерационным вычислениям. Рекомендуется сравнить все варианты вычислений, например, по моменту переполнения стека, о котором сообщает ПРОЛОГ-машина. При подготовке к выполнению и сдаче лабораторных работ рекомендуется ознакомиться с теоретическим понятием праворекурсивной логической программы (т.е. с хвостовой рекурсией), изложенным в [1-3, 9]. Итерационные вычисления и переменные-накопители представлены в [9]. Описание встроенных предикатов bound и free в Visual Prolog 5.2 в [3].

Содержание отчёта

1) заголовок лабораторной работы;
2) цель лабораторной работы;
3) индивидуальное задание на выполнение лабораторной работы;
4) математические формулы, иллюстрирующие решение задачи;
5) тексты оптимизированных логических программы;
6) результаты оптимизации логических программ;
7) выводы.

Работа № 7. **Ядро программы символьного дифференцирования. Разработка сопровождающих программ на Visual Prolog 5.2**

Цель работы - изучение внушительных возможностей логического программирования на языке Visual Prolog 5.2 при решения задач символьной обработки информации (задач так называемого *искусственного интеллекта*) на примере разработки программы символьного дифференцирования префиксных алгебраических выражений. Разработка программ, позволяющих преобразовывать результаты символьного дифференцирования.

Общие сведения

Задача *символьного дифференцирования* алгебраических функций – одна из первых задач из области так называемого искусственного интеллекта, относящихся к классу задач символьной обработки информации.

При символьном дифференцировании на входе преобразования символьное выражение (префиксное алгебраическое выражение), на выходе преобразования также получается символьное выражение (производная этой алгебраической функции). С позиции синтаксиса языка Visual Prolog 5.2 тип дифференцируемой функции должен быть представлен рекурсивной префиксной структурой данных.

Для выполнения символьного дифференцирования алгебраической функции следует представить эту функций к в рекурсивной префиксной форме. Например, функция $f(x) = a \cdot x^2 + e^x + c$. Для этого следует снять приоритеты и преобразовать инфиксную форму в префиксную. После снятия приоритетов порядок операций определяется только расставленными скобками:

$$a \cdot x^2 + e^x + c \Rightarrow (((a * (x^2)) + (e^x)) + c),$$

или

$$a \cdot x^2 + e^x + c \Rightarrow (((a * (x \wedge 2)) + (e \wedge x)) + c).$$

После преобразование инфиксной формы без приоритетов в префиксную форму получается структурированное выражение с функторами:

(((a * (x ^ 2)) + (e ^ x)) + c) =>

=> plus (plus (mult (var (a), pow (var (x), i (2))), exp (var (x))), var (c)).

Исходное выражение с помощью функторов plus, mult, pow, exp, i, var преобразованное в форму, подготовленную к символьной обработке:

$$a \cdot x^2 + e^x + c \Rightarrow (+(+(* a (expt x 2)) (exp x)) c).$$

Для удобства последующей обработки символьного префиксного выражения количество аргументов у всех функторов, с помощью которых представляются арифметические операции сложения, вычитания, умножения

и деления, фиксировано и всегда равно одному или двум, например:

$$(u + v + w) \rightarrow ((u + v) + w) \text{ или } (u + (v + w)),$$

а также $$(u * v * w) \rightarrow ((u * v) * w) \text{ или } (u * (v * w)).$$

После представления дифференцируемой функции в форме рекурсивной структуры данных можно выполнять символьное дифференцирование этой алгебраической функции по заданному символьному аргументу.

Ниже представлена разработанная на Visual Prolog 5.2 программа для символьного дифференцирования алгебраической функции по символьному аргументу.

В разделе **domains** создана рекурсивная структура EXPARIANCE с функторами i, r, var, plus, minus, mult, div, pow, sin, cos, exp, ln для того, чтобы представить дифференцируемую функцию в виде символьного префиксного алгебраического выражения. Функтор i задаёт целое число типа integer, функтор r задаёт вещественное число типа real, а функтор var задаёт переменную типа symbol. Бинарные функторы plus, minus, mult, div и pow нужны для представления суммы, разности, умножения, деления и возведения в целую степень префиксных алгебраических выражений.

В разделе **predicates** объявлен предикат **diff** с тремя аргументами: дифференцируемым выражением, аргументом, по которому выражение дифференцируется, и результатом дифференцирования. Факты:

```
diff (i (_), _, i (0)).        diff (r (_), _, i (0)).        diff(var(X), X, i (1)).
```

задают производные чисел по любому аргументу, а также производную переменной X по аргументу X. Факт diff(var(X), Y, i (0)) :- X<>Y. задаёт производную одной переменной по другой переменной, не равной ей.

Рекурсивные правила реализуют производные табличных функций и производную функции возведения выражения в целую степень. Дважды рекурсивные правила выполняют дифференцирование суммы, разности, произведения и частного двух функций.

Важно заметить, что внутри логической программы diff производные табличных функций запрограммированы как производные сложной функции. Это позволяет задавать сколько угодно много раз вложенные друг в друга функции для осуществления их символьного дифференцирования.

При этом общий анализ текста программы diff демонстрирует очевидную её простоту. Простота программы показывает всю силу логического программирования, ориентированного на решение такого рода задач. Логика предикатов 1-го порядка в клаузальной форме даёт мощнейший инструмент в решении задач символьной обработки информации:

```
domains      % префиксное алгебраическое выражение
 s=symbol
 EXPARIANCE = i (integer);                  % целое число
              r (real);                     % вещественное число
              var (s);                      % переменная
              plus (EXPARIANCE, EXPARIANCE);      % сложение
              minus (EXPARIANCE, EXPARIANCE);   % вычитание
              mult (EXPARIANCE, EXPARIANCE);     % умножение
              div (EXPARIANCE, EXPARIANCE);       % деление
              pow (EXPARIANCE, integer); %возведение в целую степень
              sin (EXPARIANCE);    % синус
              cos (EXPARIANCE);    % косинус
              exp (EXPARIANCE);    % экспонента
              ln (EXPARIANCE)       % натуральный логарифм
predicates
    nondeterm diff (EXPARIANCE, s, EXPARIANCE)

clauses
    diff (i (_), _, i (0)).

    diff (r (_), _, i (0)).

    diff(var(X), X, i (1)).

    diff(var(X), Y, i (0)) :- X<>Y.

    diff (plus (U, V), X, plus (DU, DV)) :- diff(U, X, DU),
                                            diff (V, X, DV).
    diff (minus (U, V), X, minus (DU, DV)) :- diff (U, X, DU),
                                              diff (V, X, DV).
    diff (mult (U, V), X, plus (mult (DU, V), mult (U, DV))) :-
                                                    diff (U, X, DU),
                                                    diff (V, X, DV).
    diff (div (U, V), X, div (minus (mult (DU, V), mult (U, DV)), (mult (V, V))) :-
                                                          diff (U, X, DU),
                                                          diff (V, X, DV).
    diff (pow (U, N), X, mult(i(N), mult (pow (U, N1), DU))) :- N1=N-1,
                                                          diff (U, X, DU).
    diff(sin(U), X, mult (DU, cos(U))) :- diff(U, X, DU).

    diff(cos(U), X, minus (i (0), mult (DU, sin(U)))) :- diff(U, X, DU).

    diff(exp(U), X, mult (DU, exp(U))) :- diff(U, X, DU).

    diff(ln(U), X, div (DU, U)) :- diff(U, X, DU).
```

Например, продифференцировать функцию sin(3x) по x.
Функция sin(3x) преобразуется к виду: sin (mult (i (3), var(x)). Тогда запрос на взятие производной выглядит так:

goal: diff (sin (mult (i (3), var(x)), x, DF)

Результат символьного дифференцирования:

DF=mult (plus (mult (i (0), var(x)), mult (i (3), i (1))), cos (mult (i (3), var(x))))

Анализ этого выражения, преобразование его приводит к инфиксному виду, приводит к выражению (0*x+3*1)*cos(3*x). Упростив его, получим производную 3*cos(3*x), что соответствует производной sin(3x) по x.

Попытка решить задачу символьного дифференцирования на некоем императивном языке приведёт к многократному усложнению алгоритмов и к сильному увеличению объёма требуемого программного кода, так как фактически потребует создания интерпретатора ПРОЛОГа с возможностью создавать рекурсивные структуры данных.

Задания для лабораторной работы № 7

Часть 1. Расширение таблицы производных

Таблица производных, приведённая в программе diff, совсем небольшая. Расширение таблицы производных путём добавления новых функций предлагается сделать самим обучающимся, предварительно освоив то, как это сделано с вышеназванными табличными функциями.

Далее представлены алгебраические функции, которые требуется добавить в программу diff, после чего проверить правильность её работы.

	функции	функторы – имена структур в программе
1)	tg (x),	TAN(EXPARIANCE)
2)	arc sin(x),	ASIN (EXPARIANCE)
3)	arc cos(x),	ACOS (EXPARIANCE)
4)	arc tg(x),	ATAN (EXPARIANCE)
5)	sh(x),	SINH (EXPARIANCE)
6)	ch(x),	COSH (EXPARIANCE)
7)	th(x),	TANH (EXPARIANCE)
8)	ar sh(x),	ASINH (EXPARIANCE)
9)	ar ch(x),	ACOSH (EXPARIANCE)
10)	ar th(x),	ATANH (EXPARIANCE)
11)	$\log_a(x)$	LOG (EXPARIANCE, symbol)
12)	a^x	EXPT (symbol, EXPARIANCE)
13)	$u^{v(x)}(x)$	EXPT (EXPARIANCE, EXPARIANCE)
14)	$\sqrt{x}$	SQRT (EXPARIANCE)

Представленные функции sh(x), ch(x) и th(x) есть гиперболический синус, гиперболический косинус и гиперболический тангенс.

В то же время ar sh(x), ar ch(x) и ar th(x) – обратные гиперболические функции ареа-гиперсинус, ареа-гиперкосинус и ареа-гипертангенс.

Запустить представленную программу diff. Проверить правильность и эффективность её функционирования на нескольких простых примерах. Дополнить функцию diff возможностью дифференцировать другую элементарную функцию (дополнить таблицу производных ещё одной функцией в соответствие с вариантом задания). В предлагаемой программе diff рассмотрены производные только шести элементарных функций: x^n, sin(x), cos(x), e^x и ln(x).

Часть 2. Разработка и применение рекурсивных программ для упрощения префиксных алгебраических выражений

Разработать рекурсивную функцию reduce, выполняющую редукцию (упрощение) выражений, представляющих собой результат символьного дифференцирования. Это одно из упрощений, представленных далее:

f * 1 = f, 1 * f = f, f + 0 = f', 0 + f = f, f – 0 = f, f * 0 = 0, 0 * f = 0, f / 1 = f, f / f = 1, $f^1 = f$, $f^0 = 1$, $\log_f(f) = 1$, $\log_f(1) = 0$, и т. Д. ,

где f – произвольная алгебраическая функция.

Эти же упрощения в префиксной форме будут иметь вид:

1) mult (f, i(1)) => f,
2) mult (i(1), f) => f,
3) plus (f, i(0)) => f,
4) plus (i(0), f) => f,
5) minus (f, i(0)) => f,
6) minus (f, f) => i(0).
7) mult (f, i(0)) => i(0),
8) mult (i(0), f) => i(0),
9) div (f, i(1)) => f,
10) div (f, f) => i(1),
11) div (i(0), f) => i(0),
12) pow (i(1), f) => i(1),
13) pow (f, i(0)) => i(1),
14) pow (i(0), f) => i(0),
15) pow (f, i(1)) => f,
16) log (f, f) => i(1),
17) log (i(1), f) => i(0),
18) div (sin (f), cos (f)) => tg (f),
19) plus (pow (sin (f), i(2)), pow (cos (f), 2)) => i(1),
20) div (cos (f), sin (f)) => ctg (f),
21) div (i(1), sin (f)) => cosec (f),
22) div (i(1), (cos f)) => sec (f),
23) mult (tg (f), ctg (f)) => i(1).

Здесь f – произвольное структурированное выражение типа ранее названного EXPARIANCE. Важно знать, что упрощения должны работать для любых видов функции f, например, в 1-м варианте следует так, чтобы

выполнялось преобразование:

mult (mult (sin (var (u)), i (1)), i (1)) ➔ sin (var (u)).

Запрограммировать упрощения в соответствие с вариантом задания.

Разработать факты и правила, заменяющие в любом месте префиксное алгебраическое выражение на его упрощённый аналог.

Требуемая функция должна выполнять упрощения повсюду внутри заданного префиксного скобочного, сколь угодно глубоко вложенного выражения, без пропусков упрощаемых синтаксических конструкций.

Методические указания

Добавление всех незаданных алгебраических функций к правилам diff в первой части делает программу diff универсальной. Создание правил для упрощений reduce позволяет радикально упростить результаты символьного дифференцирования. Так как программа diff позволяет находить не только производные функций одного аргумента, но и частные производные функций, то программу символьного дифференцирования можно применить, например, для проверки решений уравнений Навье-Стокса или уравнений Максвелла.

Содержание отчёта:

1) заголовок и цель лабораторной работы;
2) индивидуальное задание на выполнение лабораторной работы;
3) математические формулы, иллюстрирующие решение задачи;
4) тексты логических программы;
5) результаты оптимизации логических программ;
6) выводы.

Работа № 8. **Программирование решателей локальных задач**

Цель работы - изучение возможностей логического программирования при создании решателей локальных геометрических задач.

Общая часть

Применение логического программирования в задачах так называемого искусственного интеллекта можно также рассмотреть на примере разработки решателей локальных геометрических задач. Локальными в данной трактовке называются задачи, в которых используется, как правило, одна теорема или одна аксиома, представленные правилом в логической программе. Остальная информация, изложенная в условиях задачи, представляется в логической программе фактами. При необходимости в программу могут добавляться и другие правила, расширяя возможности решателя локальной задачи с точки зрения практики. Такой подход позволяет по-новому взглянуть на логические программы и на геометрические отношения, оставаясь в рамках небольших задач. Для разработки логических программ вербально поставленную задачу следует переформулировать на языке клаузальной формы логики [4].

Фраза «*Всякий квадрат является прямоугольником*» есть утверждение: «Для всех F: если F - квадрат, то F - прямоугольник», выраженное клаузой:

rectangle (F) if square (F).

Высказывание «*Всякий квадрат является ромбом*» есть утверждение: «Для всех F: если F - квадрат, то F - ромб», представленное клаузой

rhomb (F) if square (F).

«*Всякая фигура, являющаяся и ромбом, и прямоугольником, есть квадрат*» иначе «Для всех F: если F – ромб и F – прямоугольник, то F – квадрат»:

square (F) if rectangle (F) and rhomb (F).

«*Всякая фигура, являющаяся ромбом, или являющаяся прямоугольником, является параллелограммом*» переделывается в: «Для всех F: если F – ромб или F – прямоугольник, то F - параллелограмм», выраженное клаузой:

parallelogram (F) if rhomb (F) or rectangle (F).

Фраза «Окружность есть частный случай эллипса» есть «Для всех F: если F - окружность, то F - эллипс»: ellipse (F) if circle (F). Добавим факты вроде: circle (“C1”). rhomb (“SPQR”). rectangle (“EFGH”). square (“ABCD”).

Пример 11. «Геометрические фигуры».

```
domains s = string
database p(s)
predicates
 circle (s) ellipse (s)
 nondeterm rhomb (s)
 nondeterm rectangle (s)
 nondeterm square (s)
 nondeterm parallelogram (s)
clauses
 rectangle ("EFGH").  rectangle ("QRST").
 rectangle (F) if square (F).

 rhomb ("SPQR").  rhomb ("QRST").
 rhomb (F) if square (F).

 square ("ABCD").
 square ("MNOP")  :-  !.
 square (F) if rectangle (F) and rhomb (F).

 parallelogram (F) if rhomb (F)
                      or rectangle (F).

 ellipse (F) if circle (F).
 circle ("C1").

goal square ("QRST").
Yes
goal parallelogram (S), not(p(S)), assert(p(S)).
S=QRST
S=SPQR
S=ABCD
S=MNOP
S=EFGH
5 Solutions
```

Полученной программе можно задавать желаемые вопросы.

В примере 11 показана программа (решатель локальной задачи) на языке Visual Prolog 5.2, которая может правильно ответить, например, на вопрос «QRST – ромб?», заданный как **goal** rhomb ("QRST"). И, например, на вопрос «Какие фигуры - эллипсы?», заданный как **goal** ellipse (F). Этот элементарный пример содержит тонкие места, что расширяет круг решаемых локальных задач. В тексте этой программы формально рекурсивных клауз нет. Однако есть скрытая рекурсия. Клауза square (F) if rectangle (F) and rhomb (F). в теле содержит предикаты rectangle и rhomb, а клаузы rectangle (F) if square (F). и rhomb (F) if square (F). в теле содержат предикат square. Алгоритм резолюции при переходе к клаузе с головой square снова придёт к предикату square. Также и при переходе к клаузам с головами rectangle и rhomb алгоритм резолюции вернётся в теле этих клауз к предикатам rectangle и rhomb. Это рекурсивные вызовы. Если не принять мер, то при вопросе к программе вроде square (X), программа выдаст сообщение об ошибке **PROGRAM ERROR. 1010 (**означающая переполнение стека: *Stack overflow***)**. Переполнение стека происходит из-за бесконечной рекурсии. Для преодоления бесконечной рекурсии, как было сказано в лабораторной работе №3, необходимо сделать точку останова рекурсии. Для этого факт square (“MNOP”). следует превратить в правило square(“MNOP”):-!. Отсечение, рассмотренное в лабораторной работе № 5, остановит поиск всех решений алгоритмом резолюции среди клауз с головой square и прервёт рекурсию. Вопрос **goal** square ("QRST"). не создаст бесконечной рекурсии, а получит ответ Yes от ПРОЛОГ-машины (хотя в программе есть лишь факты rhomb("QRST"). и rectangle ("QRST").). Это очень важный приём логического программирования, проиллюстрированный на примере работы решателей локальных геометрических задач.

Другой очень важный приём логического программирования также представлен в примере 11. Ответ на вопрос **goal** parallelogram(S). Будет такой:

```
goal parallelogram(S).
S = QRST
S = SPQR
S = ABCD
S = MNOP
S = EFGH
S = QRST
S = ABCD
S = MNOP
8 Solutions
```

Чтобы устранить повторы в ответах следует использовать приём с динамическими предикатами внутренних баз фактов, предложенный в [1-2] и

подробно рассмотренный в лабораторной работе № 6. То есть нужно объявить динамический предикат p(s) в разделе **database** (то же, что раздел **facts** [3]). При добавлении каждого нового факта алгоритм резолюции запоминает все текущие полученные ответы как часть пути поиска решения, что позволяет избежать повторов в ответах [1-2]. Раздел **database** должен быть между разделом **domains** и разделом **predicates**, так как здесь тоже объявляются предикаты: **database** p(s). s – строковый тип string, значащий всё, что заключено в кавычки “”. Вопрос к логической программе с ответами без повторов выглядит так: **goal** parallelogram(S), not(p(S)), assert(p(S)). Его можно вербально сформулировать так: «Цель истинна (успешна), если S – параллелограмм, и если ещё нет динамического факта p(S), то добавить факт p(S)». Предикат assert (*Fact(.)*) истинен (true) после добавления в программу динамического факта *Fact (…).*

Оба показанных приёма широко используются при решении многих других локальных геометрических задач. К этим приёмам следует добавить новые приёмы, необходимые при использовании более сложных ситуаций в геометрических задачах, когда недостаточно унарных предикатов.

Над основными понятиями геометрии (точка, прямая, отрезок, угол, луч и т.д.) устанавливают определённые отношения: *принадлежит, имеет, пересекаются, параллельны, перпендикулярны, совпадают, скрещиваются, равны, подобны* и т.п. Говорят: «Точка принадлежит прямой», «Плоскости параллельны», «Прямые перпендикулярны», «Треугольник имеет прямой угол», «Отрезки равны», «Треугольники подобны», «Фигуры совпадают», «Прямые скрещиваются», «Прямая пересекается с плоскостью» и пр. и пр. Так как в этих примерах речь идёт об отношениях над двумя однородными аргументами (две плоскости, две прямые, два отрезка, два треугольника и т.п.) или над двумя разнородными аргументами (треугольник и угол, прямая и плоскость, точка и прямая и т.д.), то это *бинарные* отношения [12].

Бинарные отношения в исчислении предикатов 1-го порядка выражены предикатом от двух аргументов: p (A, B). Предикат papa (A, B), например, означает, что *субъект* A является отцом *объекта* B, а не наоборот. Так точно отношение параллельности parallel (a, b) не то же, что parallel (b, a), хотя с позиции геометрии это одно и то же отношение.

Бинарное отношение R на множестве M может обладать различными свойствами, среди которых важно выделить симметричность, рефлексию и транзитивность [12]:

симметричность ∀a, b ϵ M aRb => bRa,
рефлексия ∀a ϵ M aRa,
транзитивность ∀a, b, c ϵ M aRb и bRc => aRc.

Свойством *симметричности* обладают геометрические отношения равенства, совпадения, подобия, параллельности, пересечения, скрещивания и перпендикулярности.

Для двух произвольных несовпадающих точек A и B могут быть введены понятия: «прямая (AB), проходящая через точки A и B» и «отрезок [AB], соединяющий эти точки». Тогда бинарные отношения «прямая» и «отрезок» также будут симметричны, ведь прямая (AB) есть то же самое, что и прямая (BA), а отрезок [AB] есть то же самое, что и отрезок [BA].

Отношения равенства, совпадения, подобия, параллельности, отношения «иметь» и «принадлежать» являются *рефлексивными*.

Отношения равенства, совпадения, подобия, параллельности, «иметь» и «принадлежать» являются ещё и *транзитивными*.

Термин «равно» в данном случае синоним термина «конгруэнтно», когда две фигуры равны не только численно, а совпадают при наложении [12]. Близкие по смыслу отношения «равно» и «совпадает» различны с той точки зрения, что отношение «совпадает» не предполагает предварительного движения фигур для наложения.

Пример 12. Пересечение прямой и плоскости в точке: intersection (A, B, C).

```
domains c=char s=symbol st=string
predicates intersect (s, st)
 nondeterm belong (c, s)
 nondeterm belong (c, st)
 nondeterm intersection (s, st, c)
clauses intersect (a, "Alpha").
 belong ('A', a). belong ('A', "Alpha").
 intersection(A, B, C):-intersect(A, B),
          belong (C, A), belong (C, B).
goal belong (Subj, Obj).
Subj=A Obj=a
Subj=A Obj=Alpha
goal intersection (X, Y, Z).
X=a Y=Alpha Z=A
```

В примере 12 приведено решение локальной геометрической задачи об определении точки пересечения прямой и плоскости. При создании программы выбраны встроенные типы: **char** (точки ‘A’,‘B’, ...) **symbol** (прямые a, b, c, …), **string** (плоскости “Alpha”, “Beta”, “Gamma”, …). Созданы предикаты: intersect (symbol, string) означает «пересекаются прямая и плоскость», intersection (symbol, string, char) означает «пересекаются прямая и плоскость в точке», belong (char, symbol) означает «принадлежит точка прямой», а предикат belong (char, string) означает «принадлежит точка

плоскости». Единственное правило в этой программе отражает связь между геометрическими фигурами A, B и C: «Прямая A пересекает плоскость B в точке C, если и прямая A пересекается с плоскостью B, и точка C принадлежит прямой A, и точка C принадлежит плоскости B.»:

intersection (A, B, C) :- intersect(A,B), belong(C,A), belong(C,B).

Факты intersect (a, "Alpha"). belong ('A', a). belong ('A', "Alpha"). означают: «прямая a пересекается с плоскостью Alpha», «точка A принадлежит прямой a» и «точка a принадлежит плоскости Alpha» соответственно. Вопрос к программе «Что чему принадлежит?» **goal** belong (Subj, Obj). даёт верные ответы: «A принадлежит a» и «A принадлежит Alpha». А вопрос «Какая прямая с какой плоскостью и в какой точке пересекается?», заданный в виде **goal** intersection (X, Y, Z). Даёт верный ответ X = a, Y = Alpha, Z = C. Это означает «прямая a, плоскость Alpha, точка C».

Пример 13. Локальная задача на аксиому параллельных.

```
domains s = symbol c = char
predicates
        nondeterm par (s, s)
        nondeterm belong (c, s)
        nondeterm math (s, s)
clauses
 belong ('A', b). belong ('A', c).
 par (b, a). par (c, a).
 par (a, b). par (a, c).
 math (B, C):-belong (D, B), belong (D, C),
    par(B, A), par(C, A), not(belong(D, A)).
goal math (b, c).
Yes
goal math (Subj, Obj).
Subj = b Obj = b
Subj = b Obj = c
Subj = c Obj = b
Subj = c Obj = c
4 Solutions
```

В примере 13 запрограммирована локальная геометрическая задача на пятый постулат Евклида: «*Через точку, не лежащую на прямой, можно провести только одну прямую, параллельную данной прямой*», который можно переформулировать так: «*Если через точку, не лежащую на прямой, проходят две прямые, параллельные данной, то эти прямые совпадают*».

Выбраны тип s = symbol для прямых и тип c = char для точек. Предикат принадлежности точки прямой belong такой же, как в примере 12. Предикат par (s, s) означает параллельность (parallel) первого аргумента второму, а предикат math (s, s) значит совпадение (mathes) первого аргумента со вторым.

В примере 13 решается задача: «Точка A принадлежит прямым b и c. Прямые b и c параллельны прямой a. Точка A не принадлежит прямой a. Верно ли, что прямые b и c совпадают?» Запрограммированное правило math (B, C) позволяет не только получить утвердительный ответ на вопрос, но и ответить на более общий вопрос «Какие прямые совпадают?». Согласно ответам на такой вопрос, отношение math (B, C) симметрично и рефлексивно. Если нужно сделать это отношение нерефлексивным, следует к правилу math (B, C) добавить конъюнктивно A <> B.

Пример 14 посвящён определению параллельных прямых parallel (X, Y). Предикат par (s, s) нужен для создания фактов вроде par (a, b). Реализованы свойства рефлексии, транзитивности и симметрии правилами parallel0(X, Y), parallel1(X, Y) и parallel2(X, Y), соответственно. Факты о прямых par (A, B) не могут быть избыточны, чтобы не было бесконечной рекурсии. Правильно, если каждая прямая не более одного раза будет субъектом и/или объектом отношения par (A, B), а все параллельности прямых можно будет выразить орграфом [12]. Каждая дуга такого орграфа должна начинаться в субъекте и заканчиваться в объекте, а орграф должен состоять из цепочек без циклов.

Программа в примере 14 позволяет проверить наличие или отсутствие параллельности между двумя конкретными прямыми и определить все пары параллельных прямых. В ней реализованы логические свойства отношений, общие для различных геометрических объектов. Программа может быть легко переделана для исследования других геометрических отношений, обладающих такими же логическими свойствами. Чтобы программа исследовала параллельность плоскостей, достаточно изменить тип symbol на string для правильного именования плоскостей. Отношения равенства, совпадения и подобия также имеют свойства симметрии, рефлексии и транзитивности. Вместо par и parallel надо создать предикаты: math и matching (совпадение), equal и equality (равенство), similar и similarity (подобие) с аргументами соответствующих типов.

В примере 15 решена задача распознавания одинаковых треугольников. У треугольников разный порядок перечисления вершин, либо разные имена. Показанные решения локальных геометрических задач с помощью логического программирования важны для понимания геометрии с точки зрения исчисления предикатов и для создания программ на доказательство и поиск.

Пример 14. Локальная задача на свойство «параллельность».

```
domains s=symbol
database pd(s)
predicates
 nondeterm par (s)
 nondeterm par (s, s)
 nondeterm parallel0 (s, s)
 nondeterm parallel1 (s, s)
 nondeterm parallel2 (s, s)
 nondeterm parallel (s, s)
clauses
 par (a, b). par (b, c). par (c, d).
 par (A) :- par (A, _)
          ;  par (_, A).
 parallel0 (A, A) :-
    par(A), not(pd(A)), assert(pd(A)).
 parallel1 (A, B) :- par (A, B), A<>B.
 parallel1 (A, B) :- par (A, C),
                     parallel1 (C, B), A<>B.
 parallel2 (A, B) :- par (B, A), A<>B.
 parallel2 (A, B) :- par (B, C),
                     parallel2 (A, C), A<>B.
 parallel (X, Y) :- parallel0(X, Y);
                    parallel1(X, Y);
                    parallel2(X, Y).
goal parallel (d, a).
Yes
```

```
goal parallel (X, Y).
X=a Y=b
X=b Y=c
X=c Y=d
X=a Y=c
X=a Y=d
X=b Y=d
X=a Y=a
X=b Y=b
X=c Y=c
X=d Y=d
X=b Y=a
X=c Y=b
X=d Y=c
X=c Y=a
X=d Y=a
X=d Y=b
16 Solutions
```

Пример 15. Совпадающие треугольники.

```
domains c = char s = string
database d (s, s)
predicates nondeterm match (s, s)
           nondeterm triangle (s, c, c, c)
clauses triangle ("T1",'A', 'B', 'C').
        triangle ("T2",'A', 'B', 'C') :- !.
        triangle ("T1",'C', 'B', 'A').
        triangle ("T2",'C', 'B', 'A').
 triangle(N, C, A, B):-triangle (N, A, B, C).
 triangle(N, B, C, A):-triangle (N, A, B, C).
 match(N, M) :- triangle (N, A, B, C),
                triangle (M, A, B, C).
goal triangle ("T2", 'B', 'A', 'C').
Yes
goal match(N, M), N<>M, not(d(N, M)),
     not(d(M, N)), assert (d(M, N)).
N=T1 M=T2
1 Solution
```

Задания для лабораторной работы № 8

Разработать логические решатели локальных геометрических задач:
а) для проверки наличия или отсутствия параллельности между двумя конкретными плоскостями; определить все пары параллельных плоскостей;
б) для определения всех несовпадающих треугольников из имеющихся в программе данных о треугольниках; перечислить треугольники без повторов;
в) для обнаружения совпадающих и подобных геометрических фигур;
г) для проверки заданных плоскостей на предмет их пересечения;
д) для обнаружения наличия свойства пересечения, свойства скрещивания и свойства перпендикулярности у заданных в программе прямых.

Продумать возможности расширения круга решаемых локальных задач.

Методические указания

При выполнении заданий к настоящей лабораторной работе предлагается изучить принципы разработки решателей локальных геометрических задач, изложенные в [12].

Содержание отчёта

1) заголовок лабораторной работы;
2) цель лабораторной работы;
3) индивидуальное задание на выполнение лабораторной работы;
4) геометрические формулы и отношения;
5) тексты решателей геометрических задач;
6) результаты оптимизации логических программ;
7) выводы.

Библиографический список

1. *Новиков П.В.* Логическое программирование: Учебное пособие. – М.: Изд-во МАИ, 2007. – 100 с.

2. *Новиков П.В.* Учебно-методическое пособие к лабораторным работам по курсу «Логическое программирование». [Электронный ресурс]: Электронные данные и программы. - Саратов: Вузовское образование, 2017. – 103 с.

3. *Адаменко А.Н., Кучуков А.М.* Логическое программирование и Visual Prolog. – СПб.: БХВ-Петербург, 2003. – 992 с.

4. *Кириллов В.И., Старченко А.А.* Логика. – 6-е изд. – М.: Проспект, 2008. – 240 с.

5. *Шрайнер П.А.* Основы программирования на языке Пролог. – М.: Университет Информационных Технологий, 2005. – 176 с.

6. *Ковальский Р.* Логика в решении проблем. – М.: Наука, 1990. – 280 с.

7. *Мендельсон Э.* Введение в математическую логику. – М.: Наука, 1984. – 320 с.

8. *Метакидес Г., Нероуд А.* Принципы логики и логического программирования. – М.: Факториал, 1998. – 288 с.

9. *Стерлинг Л., Шапиро Э.* Искусство программирования на языке ПРОЛОГ. – М.: Мир, 1990. – 326 с.

10. *Братко И.* Алгоритмы искусственного интеллекта на языке PROLOG. – М.: Вильямс, 2004. – 640 с.

11. *Малпас Дж.* Реляционный язык Пролог и его применение. – М.: Наука, 1990.

12. *Новиков П.В.* Логическое программирование решателей для локальных геометрических задач // Современная наука: актуальные проблемы теории и практики. Серия: Естественные и Технические Науки. 2022. №10. С. 91-98. DOI 10.37882/2223-2966.2022.10.23

О Г Л А В Л Е Н И Е

Предисловие 3

Общий порядок выполнения лабораторных работ 4

Содержание отчётов по лабораторным работам……..……..……..4

Работа №1. **Основные синтаксические конструкции языка Visual Prolog 5.2**
Общие сведения..... 5
Задания для лабораторной работы № 1……...12

Работа №2. **Структурирование информации на языке Visual Prolog 5.2**
Общие сведения.....14
Задания для лабораторной работы № 2……...22

Работа №3. **Различные виды рекурсивных логических программ**
Общие сведения.....24
Задания для лабораторной работы № 3……...35

Работа №4. **Логические программы, использующие отсечение**
Общие сведения.....39
Задания для лабораторной работы № 4……...44

Работа №5. **Динамические предикаты внутренних баз данных**
Общие сведения.....46
Задания для лабораторной работы № 5……...49

Работа №6. **Оптимизация логических программ на Visual Prolog 5.2**
Общие сведения.....50
Задания для лабораторной работы № 6……..62

Работа №7. **Ядро программы символьного дифференцирования. Разработка сопровождающих программ на Visual Prolog 5.2**
Общие сведения.....64
Задания для лабораторной работы № 7……...67

Работа №8. **Программирование решателей локальных задач**
Общие сведения.....69
Задания для лабораторной работы № 8……...77

Библиографический список ……..…….....78

Новиков Павел Владимирович

ЛОГИЧЕСКОЕ ПРОГРАММИРОВАНИЕ на Visual Prolog 5.2

Редактор *Л.А. Митина*

Компьютерная вёрстка *П.В. Новикова*

Подписано в печать 20.12.2023.
Бумага офсетная. Форма 60x84 1/16. Печать цифровая.
Усл. печ. л. 4,65. Уч.-изд. л. 5,0. Тираж 100 экз.
Заказ 394/1222.

Издательство МАИ
(МАИ), Волоколамское ш., д.4,
Москва, А-80, ГСП-3 125993

Типография Издательства МАИ
(МАИ), Волоколамское ш., д.4,
Москва, А-80, ГСП-3 125993